

Modelli linguistici (LLM) personalizzati con Python di Antonio Piemontese

Ottobre 2026



Preliminari

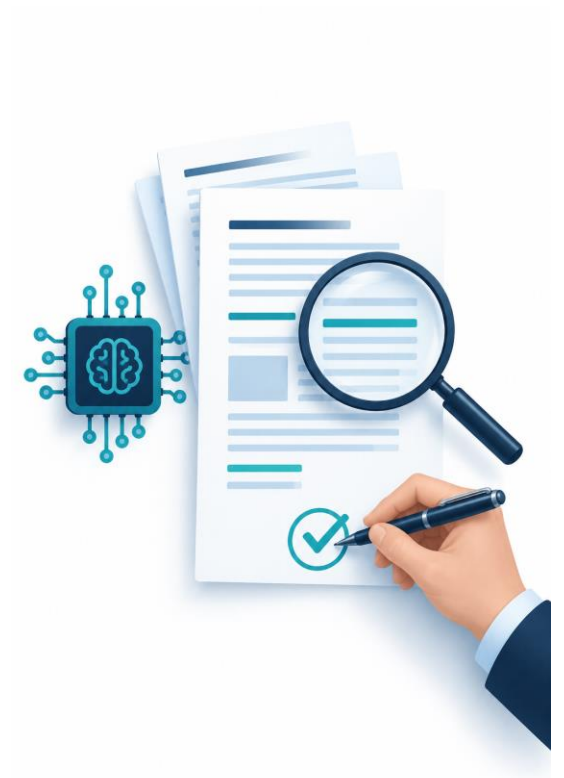
Dichiarazione di trasparenza

Uso dell'intelligenza artificiale nei materiali del corso

Parti consistenti delle slide di questo corso sono state create con strumenti di intelligenza artificiale generativa, in primo luogo ChatGPT.

Antonio Piemontese ha guidato la creazione dei materiali e ne ha verificato e affinato i contenuti attraverso successive revisioni.

Antonio Piemontese mantiene la responsabilità editoriale e didattica finale dei contenuti presentati.



Infografica riassuntiva del corso

Vincoli, approccio didattico e obiettivo

*Impara
Sperimenta
Cresci*



1 Perché il tema è complesso

Un argomento sfidante e articolato

- ✓ Argomento complesso, sia sul piano teorico sia sul piano del codice
- ✓ Molti concetti complessi
- ✓ La durata ideale del corso sarebbe di una settimana
- ✓ Python e PyTorch sono prerequisiti importanti



2 Requisiti tecnici

Strumenti e ambiente di lavoro

- ✓ Uso avanzato di PyTorch
- ✓ Necessità di CPU e anche di GPU per allenare modelli non banali in tempi ragionevoli
- ✓ Utilizzo obbligatorio di ambienti virtuali: `conda` oppure `venv`
- ✓ Obiettivo: evitare conflitti tra package e/o versioni di Python
- ✓ Se l'installazione non è possibile: uso di laboratori Godeploy



3 Vincolo di tempo

Tempi realistici e aspettative

- ✓ Un esame corsivo e attento dei notebook non è possibile in due giorni



4 Approccio didattico consigliato

Un metodo pratico e mirato

- ✓ Serve un approccio diverso da quello tradizionale
- ✓ Esame dell'approccio generale
- ✓ Esame dei concetti principali
- ✓ Esecuzione corsiva del codice, con focus solo sui passaggi chiave
- ✓ Materiale didattico: 8 notebook Jupyter molto ben commentati, in italiano, con molti link esplicativi



5 Obiettivo del corso

Competenze per il futuro

- ✓ Mettere i partecipanti in grado, successivamente al corso, di analizzare con calma, studiare e comprendere il tema
- ✓ Consentire di adattare i notebook ai propri test

*Più competenze
e più autonomia*



Obiettivo realistico: fornire basi solide e strumenti per proseguire in autonomia dopo il corso.



OGGI
LE BASI
DOMANI
PIÙ LONTANO

Costruire un LLM, da zero

Nove notebook in italiano, in puro PyTorch — dal testo grezzo di GPT-2 fino a gpt-oss

Il filo GPT-2 — costruito, pre-allenato, perfezionato · con innesti 🌱 verso gpt-oss

2

Preparare i testi

token BPE, finestra,
embedding

3

Capire l'attenzione

self-attention →
multi-head

4

Costruire il GPT

blocchi → GPTModel
124M

5

Pre-allenare

loss, training, pesi
GPT-2

6

Classificare

fine-tuning: spam
detector

7

Seguire istruzioni

Alpaca + giudice LLM

L'estensione gpt-oss — due notebook nuovi

8 **NUOVO**

Da GPT-2 a gpt-oss

RoPE · GQA · finestra · sinks · MoE —
e un mini-modello allenato da noi

9 **NUOVO**

gpt-oss in pratica

Harmony · QLoRA del 20B · il 120B
come giudice via Ollama

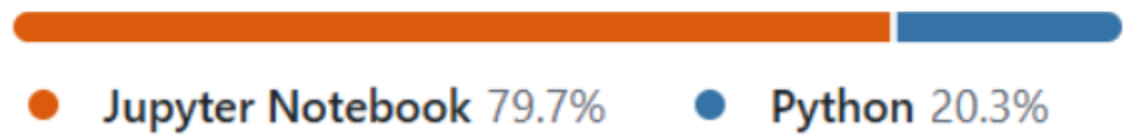
L'approccio

- Tutto da zero, in puro PyTorch: un concetto per cella
- Ispirato a Raschka, tradotto e arricchito in italiano
- GPT-2 come base: piccola, aperta, eseguibile ovunque
- gpt-oss come traguardo: innesti 🌱 prima, moduli poi

Ispirato a «Build a Large Language Model (From Scratch)» di S. Raschka · notebook 2–7 con innesti 🌱 · notebook 8–9 in preparazione

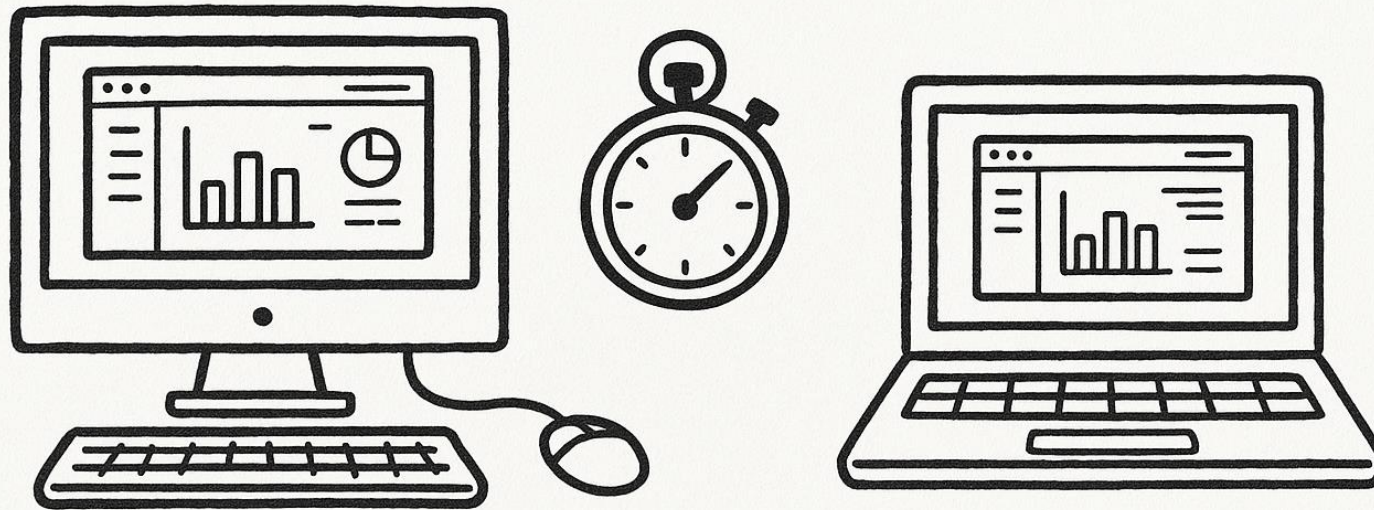
Linguaggi usati nel notebook:

Languages



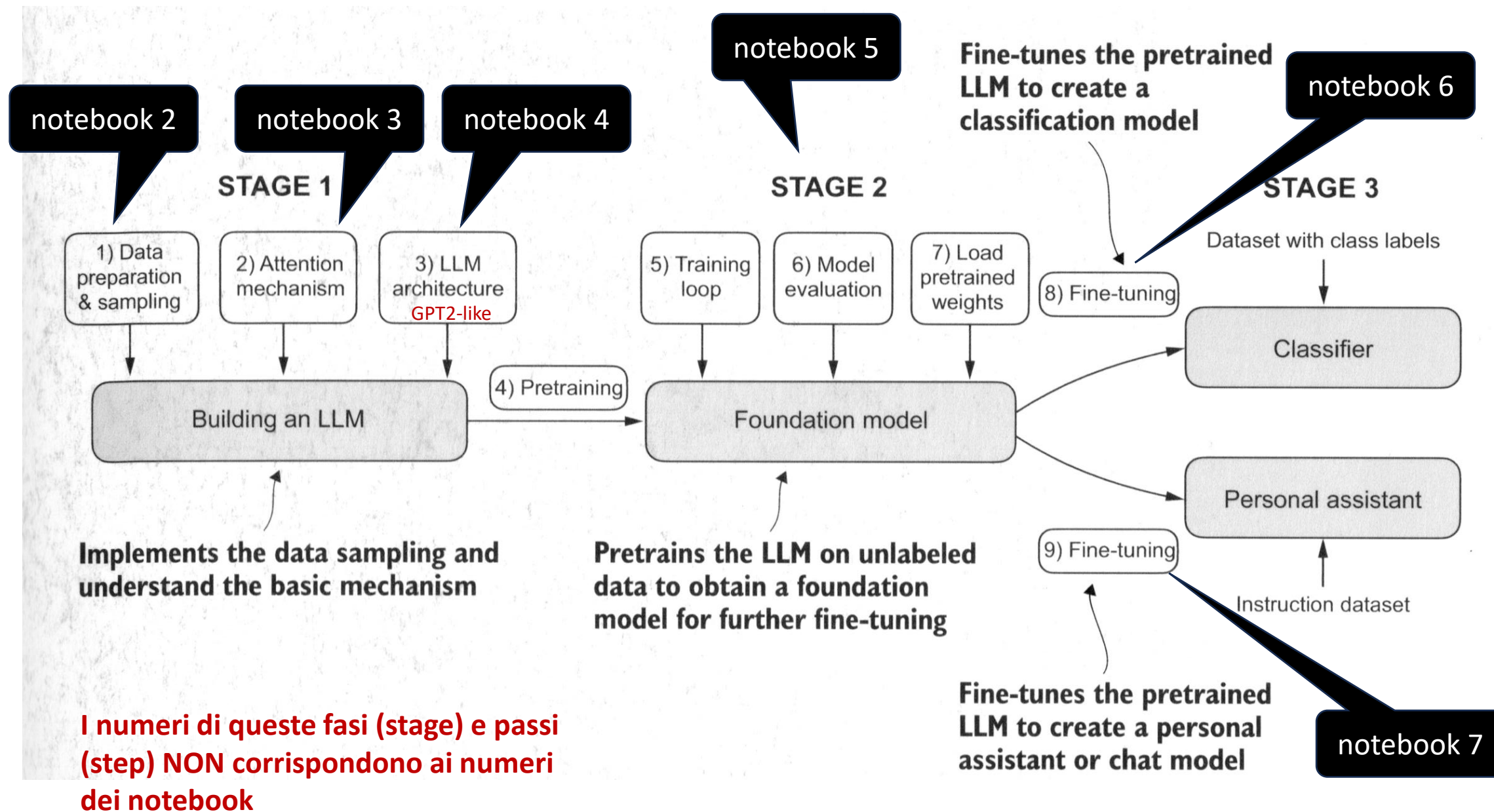
Obiettivo n.1 del corso:

Il software mostrato funziona su normali PC o laptop, anche non recenti, in tempi ragionevoli.



Saranno consegnati anche codici per utilizzi più avanzati, cioè con versioni più recenti dei modelli, con più parametri, che sfruttano le GPU.

Il programma del corso



Cosa è un notebook?

vecchio – vedi nb
A. Introduzione

1/2

Un *quaderno* con **codice** (celle) e **note** (scritte nel [linguaggio markdown](#) che è [WYSIWYM](#) e non [WYSIWYG](#)).

Le celle sono eseguibili anche **singolarmente** (ancorchè la memoria sia **condivisa**), spesso intervallate da celle di note di spiegazione ed approfondimento.

Il suffisso notebook è **.ipynb** (Interactive **PY**thon **NoteBook**), **non .py** (il codice python classico).

I notebook NON sono [IDE](#) come spyder, pycharm o VSC (per Python).

Le celle di codice eseguibili individualmente, intervallate da celle di testo, sono molto **utili per la didattica**, perché suddividono il programma in blocchi.

I principali notebook (Jupyter e Google Colab) funzionano con **3 linguaggi**: Python, R e Julia.

Google Colab

Colab sta per *colaboratory* perché il notebook è condivisibile, a differenza di jupyter notebook.

Per aziende o istituzioni, l'accesso a Google colab PUO' dover essere **autorizzato dall'amministratore**.

vecchio – vedi nb
google colab

PUNTI di FORZA:

- gira sul cloud, creando una macchina virtuale
- non richiede installazioni; occorre caricare i notebook, i file dati, le immagini ed i file PDF nel [Google drive](#) - la cartella "Colab Notebooks" presente automaticamente
- i principali package sono già installati (come anche in Anaconda/Jupyter)
- le estensioni principali (indice, variabili, tempi di esecuzione) sono fornite in modo nativo

FUNZIONAMENTO:

- Spunta verde in alto a destra quando stabilisce la connessione con l'engine in cloud (ci vuole una ventina di secondi, altrimenti allocati all'esecuzione della prima cella di codice)
- Salvataggio automatico su mydrive, cartella *colab notebooks*, presente di default.
- Avanzamento celle: shift+invio oppure ctrl+invio (come in jupyter).
- Il notebook è esportabile in un file .py con il solo codice (menù File --> Export (al fondo). A chi ha un account Google. In lettura o lettura e scrittura.
- Link, click, apre notebook, File --> Salva copia su drive.
- Output celle di codice eliminabili! 
- Condivisione nb con "condividi".
- per vedere la lista degli argomenti di una funzione/metodo - dopo aver posizionato il cursore in mezzo alle () - premere la combinazione di tasti `ctrl + maiusc + space bar` (è una funzionalità simile a `maiusc + tab` di Jupyter Notebook) - vedi in merito [questo post](#).
- per importare immagini in Google Colab vedi [qui](#) e [qui](#)

LIMITI (rispetto a Jupyter Notebook):

- *Limited Session Lifetime*: Colab sessions have a time limit, which may interrupt long-running processes
- *Dependency Installation*: Installing external dependencies may take time and needs to be done each session
- l'esecuzione delle celle di codice è più lenta per codice semplice, a causa del collo di bottiglia della rete, è più veloce per codice complesso perchè i server, le GPU e le TPU del cloud di Google sono più veloci delle normali CPU e GPU dei PC
- non ha i comandi *magic*
- i notebook devono essere modificati (rispetto alla versione del notebook per Jupyter Notebook) per quanto riguarda il caricamento delle immagini e dei PDF
- l'attivazione dei link esterni (alle URL di Internet) deve essere autorizzata esplicitamente

Architettura GPT **ricostruita** da ricercatori (Sebastian Rashcka ed altri) in base agli articoli di OpenAI

Codice Python scritto da loro (molto affidabili, sono tra i massimi esperti al mondo)

Io ho tradotto, aggiunto link note e commenti (contesto), fatto approfondimenti Python

Importante **capire bene come funziona Google Colab** (oppure **VS Code** se avete una GPU) e **PyTorch**.

Potete eseguire i notebook 2-9 in rigida sequenza (e TUTTE le celle di ognuno – è facile saltarne una) per imparare. Tuttavia, i due notebook chiave (6 e 7) sono **indipendenti** dai precedenti perché ridefiniscono tutte le classi / funzioni necessarie)

Il notebook 1 (PyTorch) è l'unico che può non essere eseguito, od essere eseguito fuori ordine.

Costruire un modello chatbot GPT2-like personalizzato vi permetterà:

- Capire come funziona (demistificarlo!)
- Risolvere i problemi di privacy e copyright
- Adattarlo ai vostri dati
- Specializzarlo sui vostri compiti (qui, classificazione, nel notebook 6, e seguire istruzioni, nel notebook 7 – due casi d'uso di amplissima applicazione)

LLM convenzionale vs reasoning model

Il corso tratta i primi

LIBRO 1 · MANNING 2024 · SI COSTRUISCE DA ZERO

Build a Large Language Model (From Scratch)

L'OUTPUT

Prompt → «Quanto fa 17 × 24?»

Risposta → «408.»

Partenza: testo grezzo, nessun modello

>

1

Tokenizzazione

Dal testo ai token con BPE (Byte Pair Encoding)

>

2

Architettura

Attenzione e transformer in stile GPT-2

>

3

Pre-addestramento

Il modello predice il token successivo

>

4

Fine-tuning

Classificazione e instruction tuning

Risultato → un LLM che segue le istruzioni e risponde subito.

LIBRO 2 · MANNING 2026 · SI PARTE DA UN MODELLO PRE-ADDESTRATO

Build a Reasoning Model (From Scratch)

L'OUTPUT

Prompt → «Quanto fa 17 × 24?»

<ragionamento> 17 × 24 = 17 × 20 + 17 × 4 = 340 + 68 = 408 </ragionamento>

Risposta → «408.»

Partenza: LLM base pre-addestrato (Qwen3 0.6B)

>

1

Base e valutazione

KV cache e verificatori: risposta giusta o no

>

2

Inference-time scaling

Chain-of-thought, self-consistency: pesi invariati

>

3

Reinforcement learning

Ricompense verificabili con GRPO

>

4

Distillazione

Il piccolo impara dalle tracce del grande

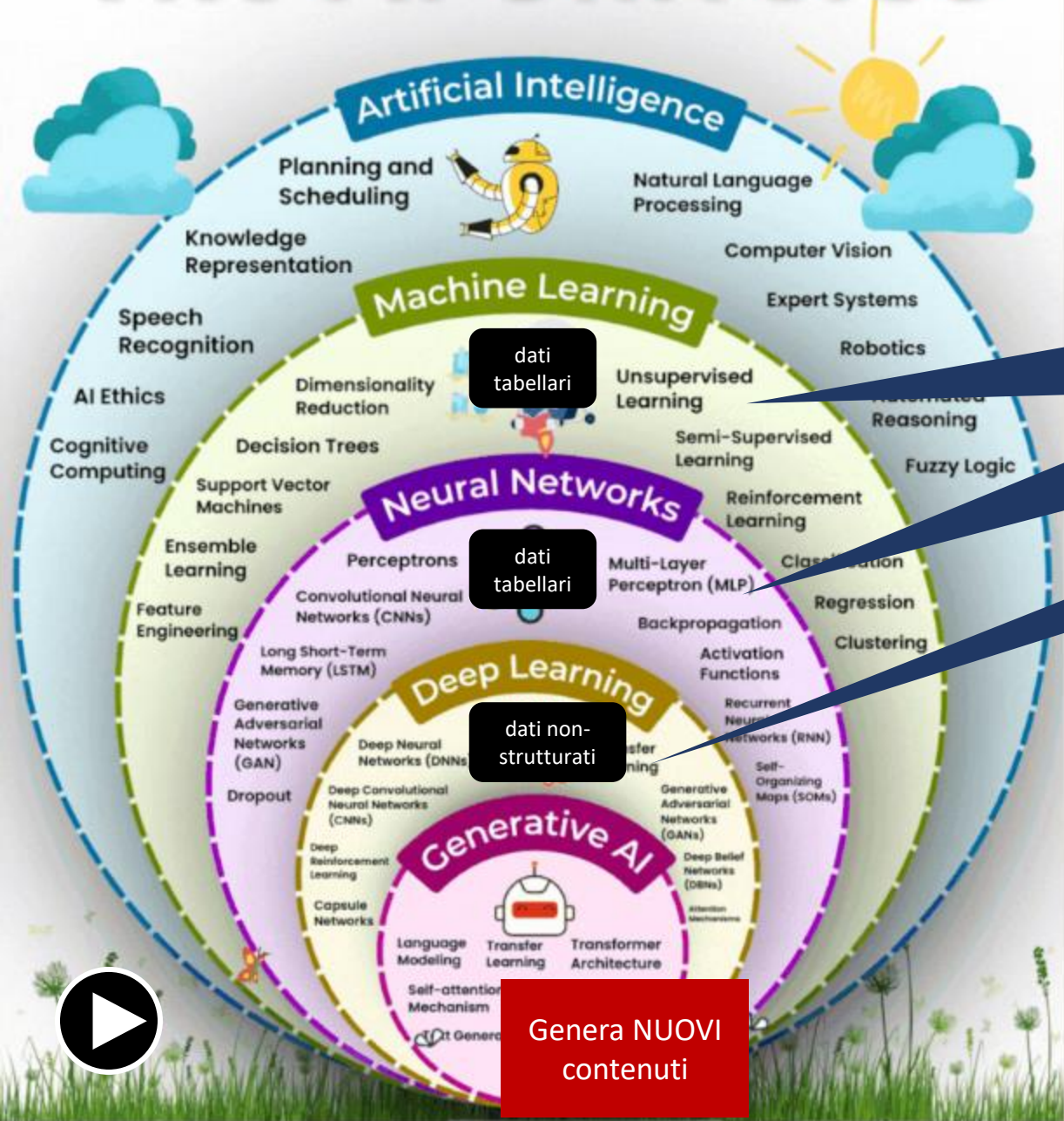
Risultato → un reasoning model che «pensa» prima di rispondere.

Le differenze in sintesi

	LLM convenzionale	Reasoning model
Punto di partenza	Testo grezzo: il modello nasce da zero	Un LLM base già pre-addestrato
Output	Risposta diretta al prompt	Catena di ragionamento, poi la risposta
Valutazione	Loss e benchmark generici	Verificatori: risposta corretta, sì o no
Dove va il calcolo	Quasi tutto nell'addestramento	Anche in inferenza (test-time compute)

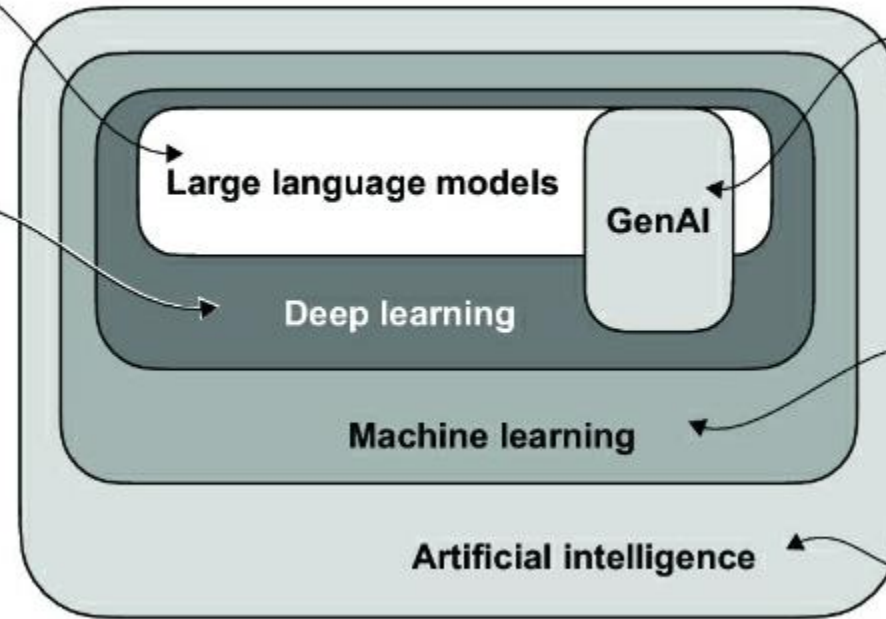
Inizio

The AI Universe



Deep neural network for parsing and generating human-like text

Machine learning with neural networks consisting of many layers



GenAI involves the use of deep neural networks to create new content, such as text, images, or various forms of media

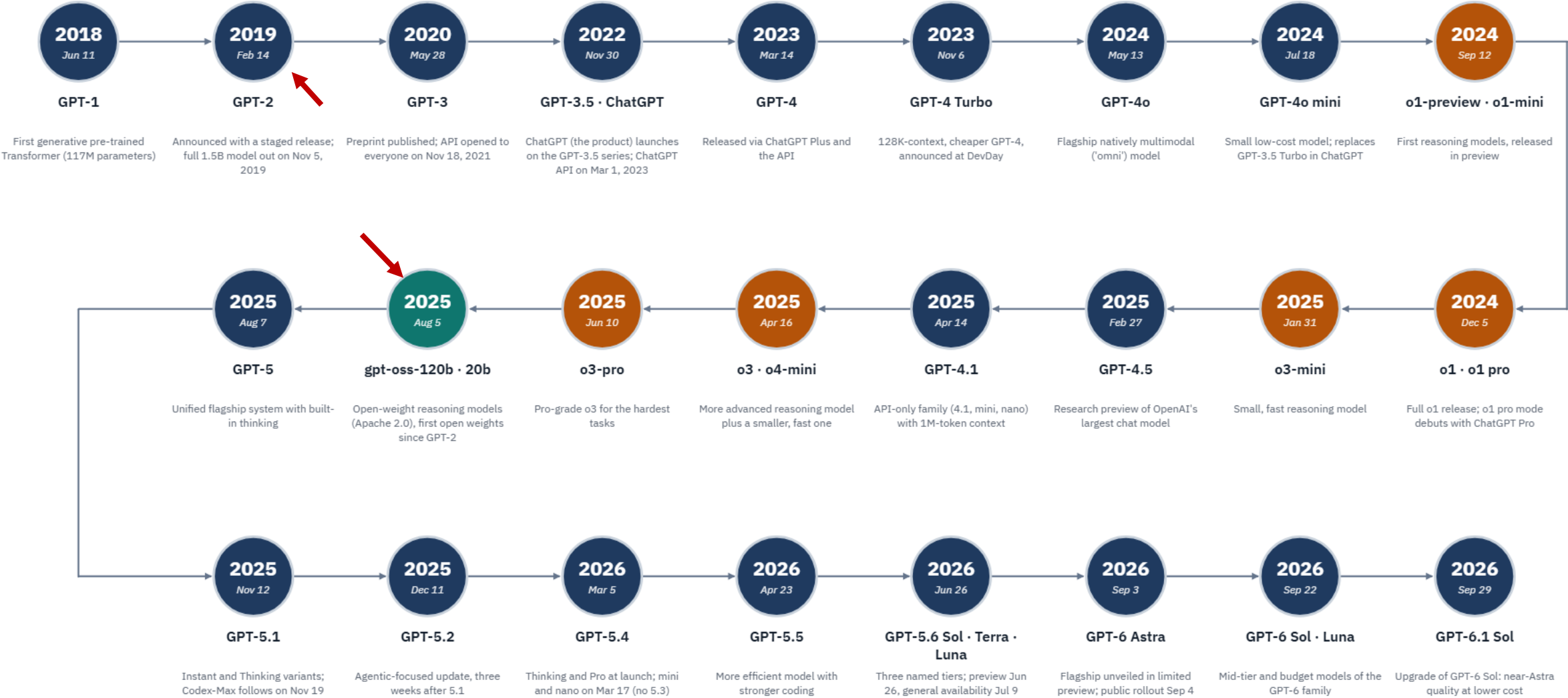
Algorithms that learn rules automatically from data

Systems with human-like intelligence

OpenAI foundation models · 2018–2026

Complete release timeline of the GPT line, the o-series reasoning models and the open-weight gpt-oss models

● GPT series ● o-series · reasoning ● Open-weight (gpt-oss)



Dates mark each model's first public availability (previews included); later general-availability dates are noted in the captions. Verified against public sources — October 2026.

Come un LLM costruisce una frase, un token alla volta

A ogni passo il modello propone una distribuzione di probabilità sul prossimo token, la temperatura la modifica e un'estrazione casuale sceglie. Il token scelto entra nel contesto e si ricomincia.

Testo

Codice

IT

EN

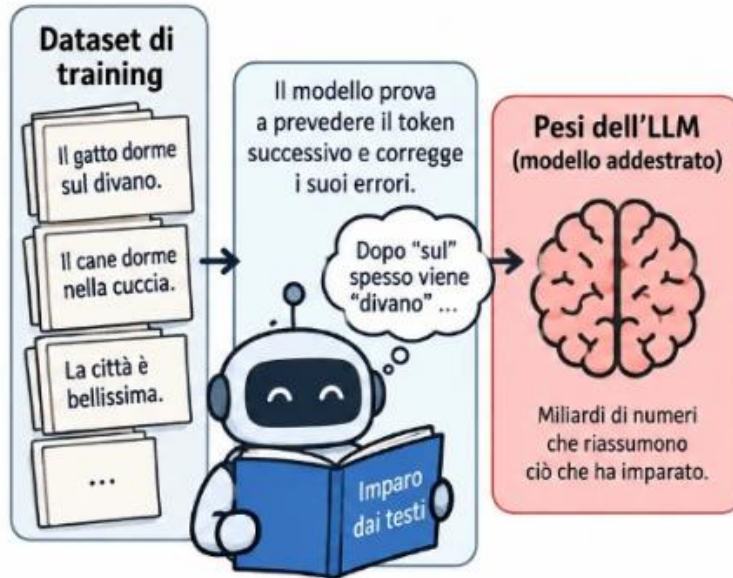
FR

[link alla app](#)

Da dove arrivano le probabilità?

1. ADDESTRAMENTO (una volta, prima dell'uso)

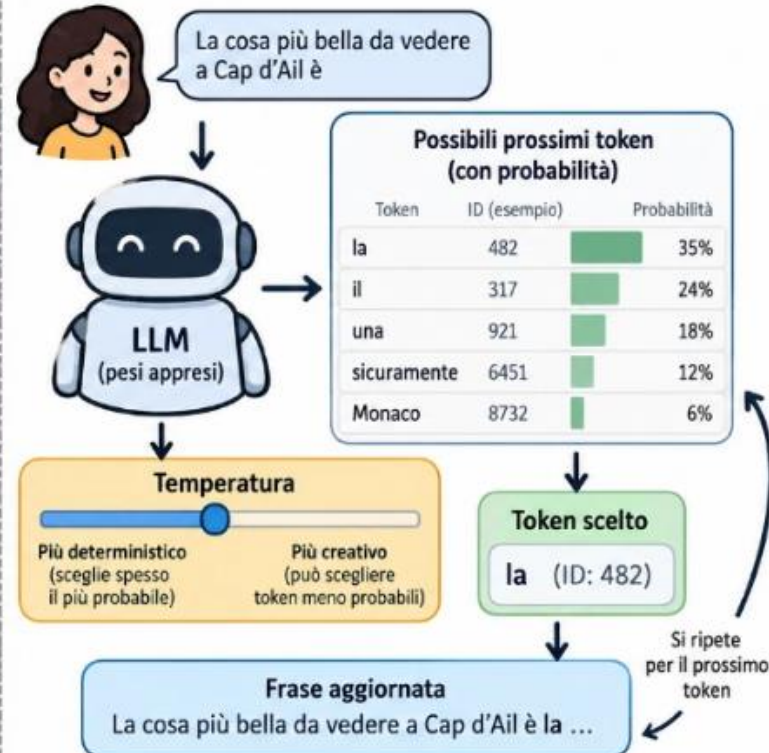
Il modello legge molti testi e impara i pattern.



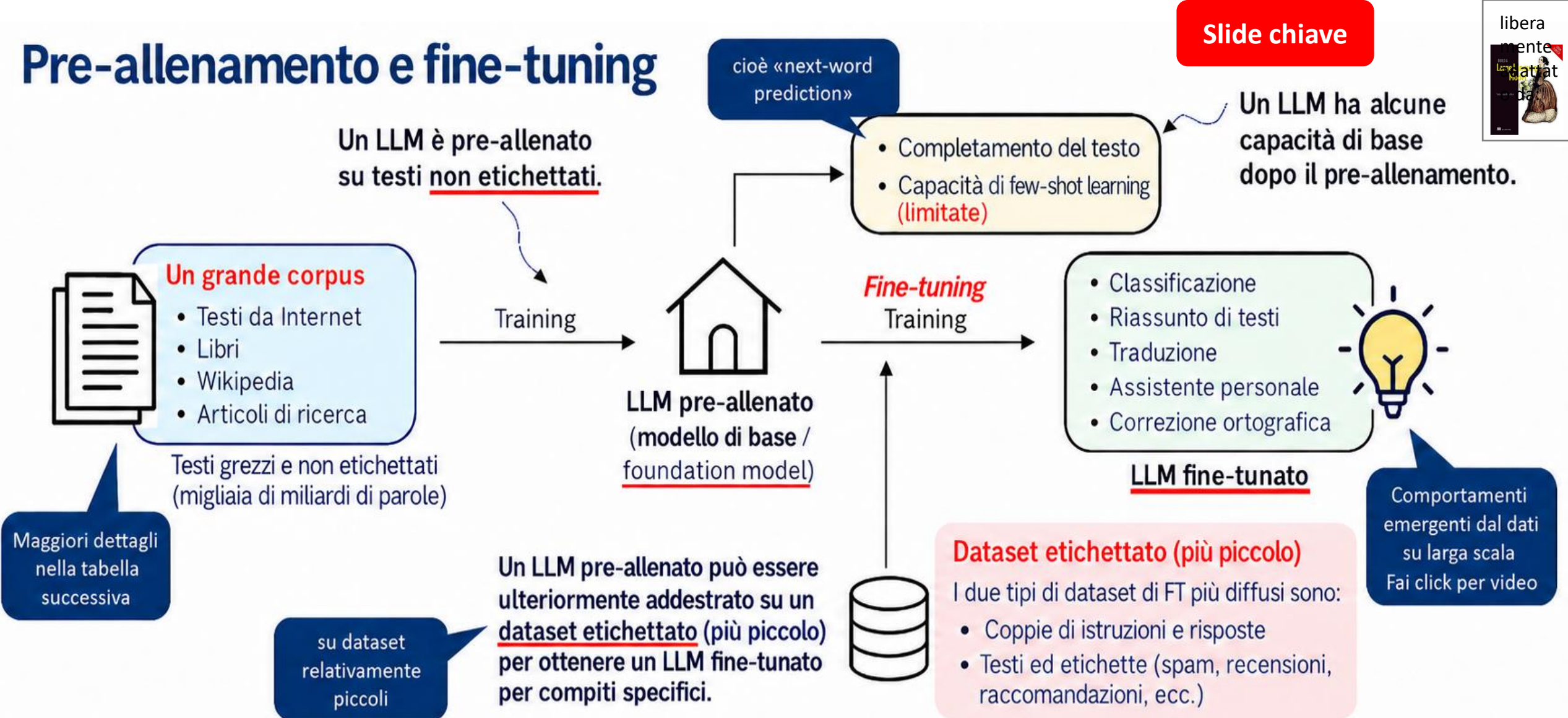
Dopo l'addestramento, il modello non conserva i testi, ma solo i suoi pesi, che codificano statisticamente i pattern appresi.

2. UTILIZZO (ogni volta che generi una frase)

Il modello usa i suoi pesi per calcolare le probabilità dei prossimi token.



Pre-allenamento e fine-tuning



➡ Il pre-allenamento di un LLM consiste nella previsione della parola successiva (next-word prediction) su grandi dataset di testi. Un LLM pre-allenato può poi essere fine-tunato utilizzando un dataset etichettato più piccolo.

Il dataset di preaddestramento del popolare LLM GPT-3

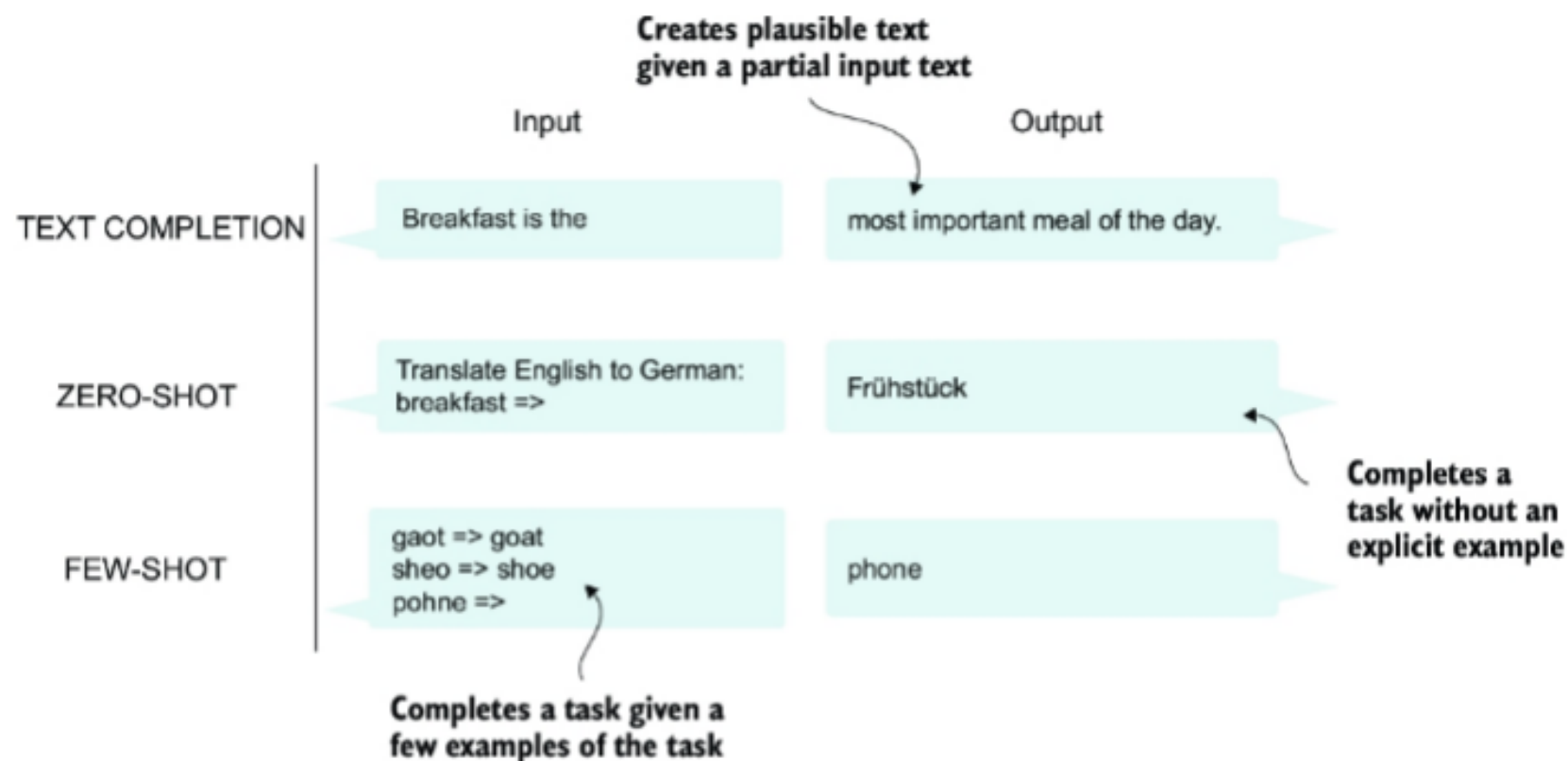
Nome del dataset	Descrizione del dataset	Numero di token	Proporzione nei dati di addestramento
CommonCrawl (filtrato)	Dati raccolti dal web	410 miliardi	60%
WebText2	Dati raccolti dal web	19 miliardi	22%
Books1	Corpus di libri basato su Internet	12 miliardi	8%
Books2	Corpus di libri basato su Internet	55 miliardi	8%
Wikipedia	Testo di alta qualità	3 miliardi	3%

DETTAGLI DEL DATASET GPT-3

La tabella qui a fianco mostra il dataset utilizzato per GPT-3. La colonna delle proporzioni nella tabella arriva al 100% dei dati campionati, al netto degli arrotondamenti. Sebbene i sottoinsiemi nella colonna Numero di token totalizzino 499 miliardi, il modello è stato addestrato solo su 300 miliardi di token. Gli autori dell'articolo su GPT-3 non hanno specificato perché il modello non sia stato addestrato su tutti i 499 miliardi di token.

Per contestualizzare, si consideri la dimensione del dataset CommonCrawl, che da solo contiene 410 miliardi di token e richiede circa 570 GB di spazio di archiviazione. In confronto, iterazioni successive di modelli come GPT-3, come LLaMA di Meta, hanno ampliato il loro ambito di addestramento includendo ulteriori fonti di dati come gli articoli di ricerca di arXiv (92 GB) e le domande e risposte di StackExchange relative al codice (78 GB).

Gli autori dell'articolo su GPT-3 non hanno condiviso il dataset di addestramento, ma un dataset comparabile disponibile pubblicamente è *Dolma: An Open Corpus of Three Trillion Tokens for LLM Pretraining Research* di Soldaini et al. 2024 (<https://arxiv.org/abs/2402.00159>). Tuttavia, la raccolta può contenere opere protette da copyright e i termini esatti di utilizzo possono dipendere dal caso d'uso previsto e dal paese.



Oltre al completamento del testo, gli LLM di tipo GPT possono svolgere **diversi compiti** in base agli input ricevuti, **senza necessità di riaddestramento, fine-tuning o modifiche** dell'architettura del modello specifiche per il compito.

Talvolta è utile fornire, all'interno dell'input, alcuni **esempi del risultato atteso**: questa modalità è nota come **few-shot**.

Tuttavia, gli LLM di tipo GPT sono anche in grado di svolgere compiti **senza che venga fornito alcun esempio specifico**: questa modalità è chiamata **zero-shot**.

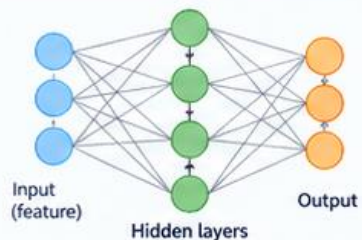
Da rappresentazioni semplici a modelli sempre più sofisticati, ognuno con punti di forza e limiti.

Prima del 2017: lo stato dell'arte delle architetture neurali

Quali architetture erano disponibili, per fare cosa, e con quali limiti?

Stesso obiettivo: capire e generare linguaggio o sequenze, con approcci diversi.

Feed-forward networks (MLP) (1986)



- usate su feature ingegnerizzate manualmente
- adatte a classificazione semplice e stima di punteggi

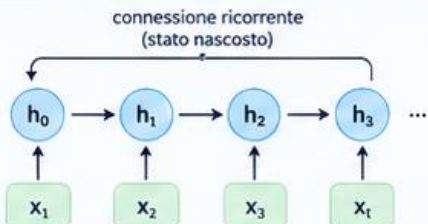
Compiti tipici:

- sentiment analysis di base
- ranking / scoring
- classificazione su feature testuali aggregate

! Limiti:

- non modellano bene l'ordine della sequenza
- dipendono molto dal feature engineering

RNN (1990)



- leggono la sequenza un elemento alla volta
- mantengono uno stato nascosto

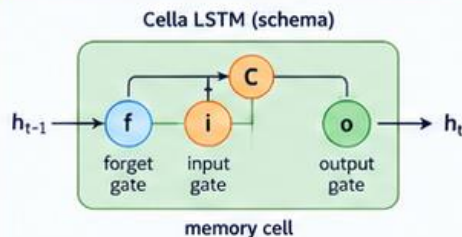
Compiti tipici:

- language modeling
- sequence labeling
- speech e serie temporali

! Limiti:

- difficoltà con dipendenze lunghe
- training instabile / vanishing gradient

LSTM / GRU (1997 / 2014)



- RNN con meccanismi di memoria più robusti
- migliori delle RNN semplici su contesti più lunghi

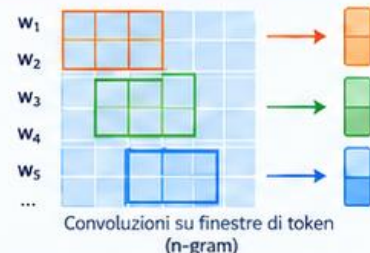
Compiti tipici:

- traduzione neurale iniziale
- speech recognition
- language modeling
- previsione su sequenze

! Limiti:

- ancora sequenziali
- parallelizzazione limitata
- costo crescente su sequenze lunghe

CNN per testo / sequenze (2014)



- estraggono pattern locali (n-gram, finestre)
- efficaci per classificazione e alcune sequenze brevi

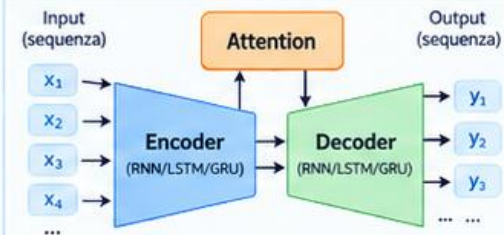
Compiti tipici:

- text classification
- sentence classification
- alcuni modelli di sequence modeling

! Limiti:

- contesto globale meno naturale
- servono più strati per dipendenze lontane

Seq2Seq encoder-decoder + attention (2014 / 2015)



- encoder legge l'input, decoder genera l'output
- l'attention aiuta il decoder a consultare parti rilevanti dell'input

Compiti tipici:

- traduzione neurale
- riassunto iniziale
- dialogo e generazione sequenza → sequenza

! Limiti:

- il decoder resta autoregressivo e sequenziale
- training e inferenza lenti rispetto a soluzioni completamente parallelizzabili
- architetture più complesse da addestrare

Che cosa si faceva prima del 2017?



Classificazione testuale
Sentiment, spam, topic.



Language modeling
Previsione della parola/token successivo.



Traduzione neurale
Soprattutto con encoder-decoder.



Speech e sequenze temporali
Riconoscimento e forecasting.

Architettura	Punto di forza	Limite chiave
MLP (1986)	semplici e veloci	ignorano l'ordine
RNN (1990)	modellano la sequenza	memoria limitata
LSTM/GRU (1997 / 2014)	migliore memoria	poco parallelizzabili
CNN (2014)	ottime sui pattern locali	contesto lontano difficile
Seq2Seq + attention (2014 / 2015)	molto utili per traduzione	ancora sequenziali



In sintesi: prima del 2017 il panorama neurale era dominato da RNN, LSTM/GRU, CNN e encoder-decoder con attention, ciascuno utile ma con limiti forti su memoria lunga, parallelizzazione o contesto globale. ←

Frase chiave da ricordare



Prima dei Transformer esistevano già modelli encoder–decoder con attention; il Transformer non introduce l’idea di attention in sé, ma costruisce un’intera architettura fondata principalmente su di essa, senza RNN come ossatura principale.

In breve

Prima del 2017, nella traduzione neurale e in altri compiti sequenza→sequenza, si usavano spesso modelli di questo tipo:

- encoder RNN / LSTM / GRU
- decoder RNN / LSTM / GRU
- meccanismo di attention tra encoder e decoder

Quindi:

l’attention non nasce con i Transformer.

Che cosa faceva questa attention “pre-Transformer”?

Serviva a evitare il vecchio problema del Seq2Seq puro.

Senza attention

L’encoder leggeva tutta la frase di input e la comprimereva in un unico vettore finale.

Questo era un **collo di bottiglia**, soprattutto per frasi lunghe.

Con attention

Il decoder, mentre generava ogni parola di output, **poteva “guardare” le varie parti dell’input** prodotte dall’encoder e dare più peso a quelle rilevanti.

Per esempio, se traduce:

- input: “This is an example”
- output parziale: “Das ist ein ...”

quando deve produrre “**Beispiel**”, il decoder può concentrarsi soprattutto sulla parte dell’input che corrisponde a “**example**”.

Quindi in che senso il Transformer è nuovo?

Il Transformer **non inventa l’attention**.

La sua novità è che:

1. **elimina la ricorrenza** come struttura principale;
2. rende l’**attention** il **meccanismo centrale** dell’architettura;
3. usa in modo sistematico la **self-attention**;
4. permette una **parallelizzazione molto maggiore**.

Distinzione utile

Per non confondere le cose:

Prima dei Transformer

c’era soprattutto:

- **attention encoder→decoder**
- cioè il decoder consulta gli stati dell’encoder

questa è molto vicina a quella che oggi chiamiamo **cross-attention**.

Nei Transformer

oltre a quella, diventa centrale anche:

- **self-attention nell’encoder**
- **self-attention nel decoder**
- e poi anche **cross-attention encoder–decoder**

2017

2017: esce il paper che cambia l'IA

Nasce il Transformer

Una nuova architettura. Un nuovo modo di capire il linguaggio. Un punto di svolta.

"Da un paper,
una nuova generazione
di possibilità."

Ideas
today
shape
a brighter
tomorrow.

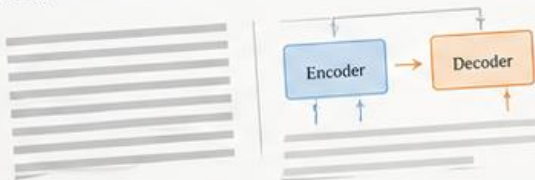
Attention Is All You Need

Ashish Vaswani Noam Shazeer Niki Parmar Jakob Uszkoreit
Llion Jones Aidan N. Gomez Lukasz Kaiser Illia Polosukhin

Google Brain

Abstract

We present the Transformer, a model architecture based solely on attention mechanisms, dispensing with recurrence and convolutions. The model achieves state-of-the-art results in sequence transduction tasks such as machine translation, while being more parallelizable than recurrent models.



1 Introduction

Large
ideas
often start
as a simple
paper.



Link all'articolo
originale



Preprint su arXiv:
12 giugno 2017



Presentato a
NeurIPS 2017
(Long Beach, USA)



La timeline



12 giugno 2017



il paper appare su arXiv

La prima versione del lavoro diventa pubblica alla comunità.



31 agosto 2017



Google Research pubblica un post divulgativo sul Transformer

Un articolo accessibile presenta le idee chiave a un pubblico più ampio.



Fine 2017



il lavoro viene percepito come una svolta per traduzione e language understanding

La comunità inizia a riconoscerne il grande potenziale oltre la traduzione automatica.



Perché è storico?

1

rende centrale la self-attention

L'attention diventa il meccanismo principale, senza bisogno di ricorrenza o convoluzioni.

2

riduce il ruolo della ricorrenza

Supera i limiti delle RNN e permette maggiore parallelizzazione.

3

diventa la base di modelli come BERT, GPT e molti sistemi moderni

Un'idea semplice ma potente, che apre la strada ai modelli di oggi.



Prime reazioni mediatiche e accademiche

“

Una nuova architettura basata solo sull'attention. ”

”



arXiv

Nota introduttiva al paper

“

Più parallelizzabile delle RNN, con prestazioni competitive. ”



Google Research Blog

31 agosto 2017

“

Risultati forti nella traduzione automatica. ”

”



Copertura accademica

Fine 2017

“

L'inizio di una nuova era per l'IA generativa. ”

”



Lettura storica successiva

Impatto nel lungo periodo



Una slide storica: il momento in cui il Transformer entra nella storia dell'IA.

Stessa domanda.
Orizzonti più lontani.



L'essenza di un Transformer

Nel 2017: architettura originale text-to-text

Dalle sequenze
al significato, insieme



Rappresentazione contestuale

È un embedding contestuale: un vettore numerico che rappresenta un token tenendo conto anche del contesto, cioè degli altri token rilevanti della sequenza.

In breve: non è solo il token, ma il token nel suo contesto.

1 Cosa fa un Transformer

L'essenza di un **Transformer** è questa:

“trasformare una sequenza di elementi in rappresentazioni contestualizzate, facendo sì che ogni elemento possa “guardare” gli altri e decidere quali sono più rilevanti.”

Il meccanismo centrale è la **self-attention**.



L'architettura originale del 2017 era

TEXT-TO-TEXT

(sequenza in ingresso → sequenza in uscita),
nata soprattutto per la traduzione automatica.

2 Un esempio concreto

Prendiamo:

“Il cane rincorre la palla”

Il Transformer non tratta “cane”, “rincorre” e “palla” come elementi isolati. Per rappresentare, per esempio, “rincorre”, considera anche gli altri token e attribuisce loro pesi diversi. In questo modo costruisce una rappresentazione di “rincorre” che dipende dal contesto specifico della frase.



3 Il processo in più strati

Quindi il cuore del Transformer è:

token iniziali → attention → rappresentazioni contestualizzate

Questa trasformazione viene ripetuta in più strati. Ogni strato raffina le rappresentazioni precedenti, facendo emergere relazioni sempre più astratte: sintattiche, semantiche e, nei modelli più grandi, anche concettuali.



4 Confronto con RNN/LSTM

La grande differenza rispetto alle vecchie RNN/LSTM è che il Transformer **non deve leggere necessariamente la sequenza uno step alla volta per costruire il contesto**. Durante il training può elaborare molti token in parallelo e mettere direttamente in relazione anche token molto lontani.

RNN/LSTM

Elaborazione sequenziale
(uno step alla volta)



Deve leggere la sequenza
uno step alla volta.

Transformer

Elaborazione in parallelo
di tutti i token



Può elaborare molti token in parallelo
e mettere direttamente in relazione
anche token molto lontani.

5 In sintesi

In forma estremamente sintetica:

“Un Transformer è una rete neurale che usa l’attenzione per trasformare elementi iniziali in rappresentazioni dipendenti dal contesto, catturando le relazioni tra gli elementi della sequenza.”

E ancora più breve, da slide:

**Transformer = rappresentazioni +
attenzione + trasformazioni successive del contesto.**



Stessa architettura. Tanti compiti. Un'unica idea: **il contesto conta.**

L'architettura Transformer comprende due sottomoduli: un encoder e un decoder. L'encoder elabora il testo di input e lo codifica in una serie di rappresentazioni numeriche, o vettori, che ne catturano le informazioni contestuali. Il decoder utilizza quindi questi vettori codificati per generare il testo di output.

In un compito di traduzione, per esempio, l'encoder codifica il testo nella lingua di origine trasformandolo in vettori, mentre il decoder decodifica questi vettori per generare il testo nella lingua di destinazione.

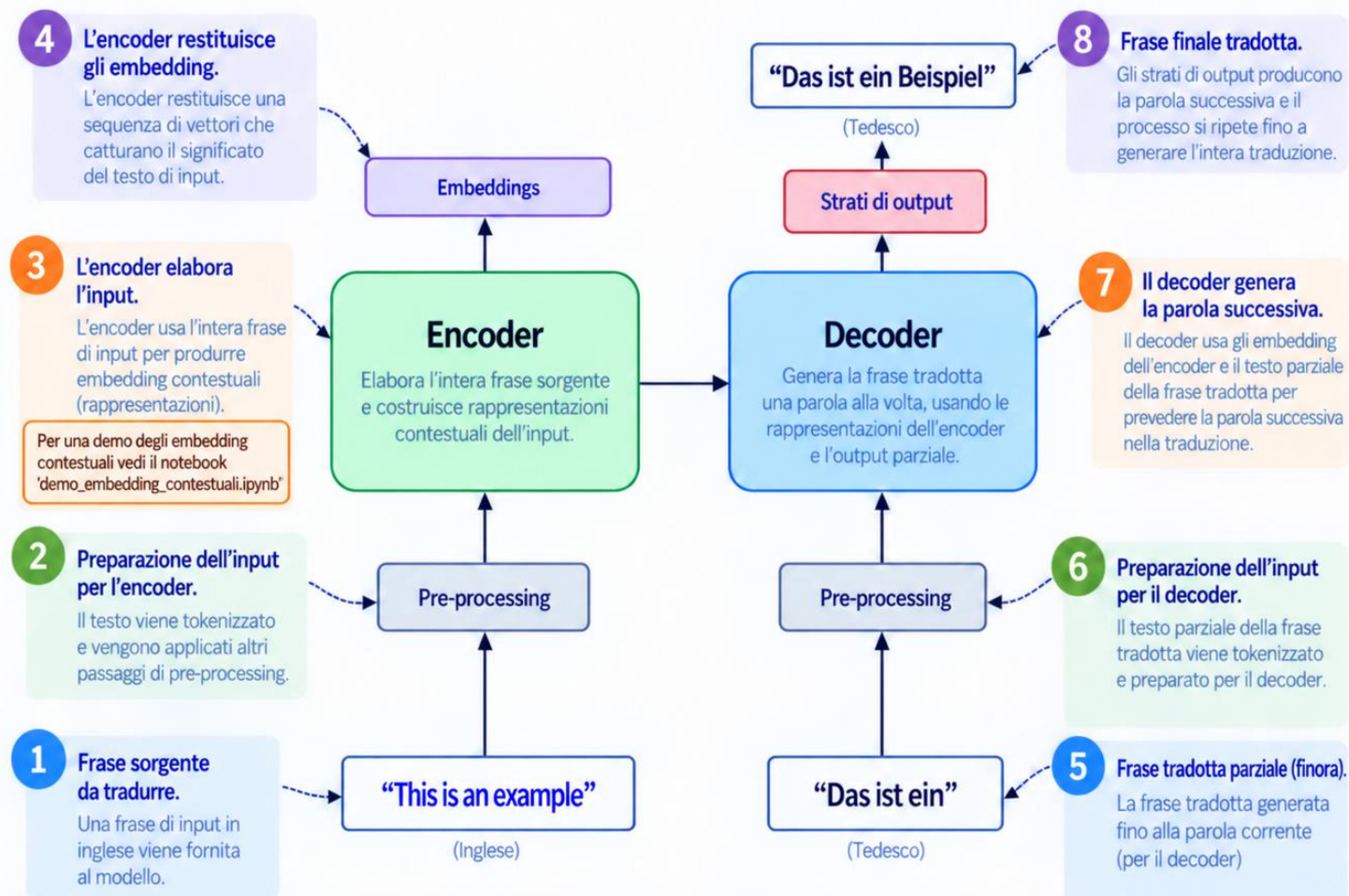
Sia l'encoder sia il decoder comprendono numerosi strati collegati da un meccanismo chiamato **self-attention**. Potrebbero sorgere molte domande su come gli input vengano preelaborati e codificati. Queste saranno...

Il testo nell'immagine inizia e termina a metà frase.

La figura illustra l'**inferenza** di un Transformer encoder–decoder **già addestrato**, cioè il suo utilizzo per produrre una traduzione. Più precisamente, mostra **un passaggio della generazione**, quello che completa «Das ist ein» con «Beispiel».

Transformer Encoder–Decoder per la traduzione automatica

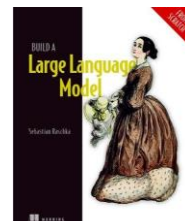
Una panoramica semplificata di come un Transformer traduce una frase dall'inglese al tedesco.



Questa è una rappresentazione semplificata dell'architettura Transformer originale per la traduzione automatica.

L'encoder elabora l'intero testo sorgente e produce embedding contestuali, mentre il decoder genera la frase tradotta passo dopo passo, usando tali embedding e le parole generate in precedenza.

liberamente
adattato da:



Una raffigurazione semplificata dell'architettura transformer originale, un modello di deep learning per la traduzione linguistica. Il transformer è composto da **due parti**: (a) un **encoder** che elabora il testo di ingresso e produce una rappresentazione a **embedding** (una rappresentazione numerica che cattura molti fattori diversi in dimensioni diverse) del testo, che il (b) **decoder** può usare per generare il testo tradotto una parola alla volta. La figura mostra la **fase finale** del processo di traduzione, in cui il decoder deve generare **soltanto l'ultima parola** («Beispiel»), dati il testo di ingresso originale ("This is an example") e una frase parzialmente tradotta ("Das ist ein"), per completare la traduzione.

Come il decoder genera la traduzione

➡ il decoder è la parte che genera il testo tradotto. Ma non lavora da solo: utilizza le informazioni sulla frase originale preparate dall'encoder. La traduzione è quindi il risultato della collaborazione fra i due componenti.

1. L'encoder prepara la frase da tradurre

Nell'esempio, l'encoder riceve:

This is an example

La trasforma in una **sequenza di vettori contestualizzati**: rappresentazioni numeriche dei token che tengono conto anche degli altri token della frase. **Non produce già la traduzione tedesca**; produce le informazioni che il decoder utilizzerà per generarla.

2. Il decoder combina due fonti di informazione *(entrambe come embedding)*

Nel momento rappresentato dalla figura, il decoder deve continuare:

Das ist ein ...

Per farlo, utilizza sia **la frase inglese codificata dall'encoder**, sia **la traduzione tedesca già generata**. Queste due fonti vengono elaborate attraverso due meccanismi distinti.

La self-attention considera il testo tedesco già prodotto. Permette di mettere in relazione i token di «Das ist ein», senza accedere ai token futuri.

La cross-attention consulta le rappresentazioni della frase inglese. Permette al decoder di combinare le informazioni provenienti dalle diverse posizioni del testo sorgente, attribuendo loro pesi diversi per la previsione corrente. È il collegamento fondamentale fra encoder e decoder.

Gli strati di uscita trasformano il risultato in probabilità sui possibili token successivi. La procedura di decodifica usa queste probabilità per scegliere come continuare. Nel percorso illustrato, la continuazione è «Beispiel». Il calcolo può essere riassunto così:

$P(\text{prossimo token} \mid \text{frase inglese, tedesco già generato})$

➡ Quindi **non sta cercando soltanto una continuazione plausibile di «Das ist ein»**: sta cercando una continuazione coerente anche con il testo inglese da tradurre.

Come lavorano self-attention e cross-attention nel decoder

Esempio: traduzione di “This is an example” → “Das ist ein Beispiel”

Il decoder riceve due input

1 Input 1: frase tradotta parziale

“Das ist ein”

(Tedesco)

embedding della frase tradotta parziale

2 Input 2: output dell'encoder

embedding contestuali della frase sorgente:

This

is

an

example

(Inglese)



Per prevedere il prossimo token, il decoder combina queste due fonti di informazione.

1 Self-attention nel decoder (solo token già generati)

“Das ist ein”

Embedding dei token della frase tradotta parziale

Das

ist

ein

Token corrente

(da cui si predice il prossimo)



La self-attention confronta i token già generati tra loro.



Serve a mantenere coerenza nella frase tradotta parziale.



Nel decoder è **mascherata**: non può vedere la parola futura “Beispiel”.

2 Cross-attention con l'encoder

Token della frase sorgente (output encoder)

This

is

an

example

Das

ist

ein

Token del decoder
(frase tradotta parziale)



La **cross-attention** fa consultare al decoder le rappresentazioni prodotte dall'encoder.



Qui, per prevedere il prossimo token, il decoder si concentra soprattutto su “example”.

3 Risultato al passo corrente

Self-attention
usa il contesto di
“Das ist ein”

+

Cross-attention
usa l'informazione rilevante da
“This is an example”



Output del decoder
prossimo token previsto:

Beispiel



In sintesi: la self-attention aiuta il decoder a capire il contesto della traduzione già prodotta; la cross-attention lo collega alla frase sorgente elaborata dall'encoder. Insieme, permettono di predire la parola successiva.

Come è stato allenato il modello (encoder + decoder)?

Addestramento del modello Transformer encoder-decoder per la traduzione automatica

1 Dati di addestramento

Molte coppie di frasi:
frase originale →
traduzione di riferimento.

This is an example



Das ist ein Beispiel

Good morning



Guten Morgen

I like apples



Ich mag Äpfel

...

Durante l'addestramento
vengono fornite molte coppie
sorgente → traduzione corretta.

2 Che cosa riceve il modello?

Il modello riceve la frase sorgente
e il prefisso corretto della
traduzione.

Frase sorgente
(in ingresso all'encoder)

This is an example

Prefisso corretto
della traduzione
(in ingresso al decoder)

Das ist ein

(teacher forcing)



Encoder
(codifica la
frase sorgente)



rappresentazione
del contesto



Decoder
(genera il prossimo
token usando
il prefisso)

3 Obiettivo del modello

Il modello è addestrato
a prevedere il token
seguente.

Prefisso in ingresso
al decoder

Das ist ein



Decoder

Prossimo token (predizione)

Beispiel	0.72	
Haus	0.10	
Tag	0.05	
...	0.03	

Token previsto: ?

Token corretto
della traduzione

Beispiel

4 Confronto con la risposta corretta

L'errore rispetto alla traduzione
di riferimento viene calcolato.

Token previsto
dal modello

Haus

Token corretto
di riferimento

Beispiel



Funzione di perdita
(es. cross-entropy)

L'errore viene usato per
aggiornare i pesi di entrambi
i componenti del modello:



Aggiorna
i pesi
dell'encoder



Aggiorna
i pesi
del decoder

L'addestramento riguarda
l'intero modello: encoder e decoder.

5 Ripetizione su molti esempi

Ripetendo questo processo
su molte coppie di frasi,
il modello completo
impara a tradurre.

Coppia 1 ...

Coppia 2 ...

Coppia 3 ...

...

Coppia N ...

Molti passi di addestramento
(su tutto il dataset).



Non è il semplice nome
'decoder' a dargli questa
capacità: la capacità di
tradurre viene appresa
dall'intero modello
attraverso l'addestramento
sul compito.



Punto chiave: il modello genera la traduzione un token alla volta, condizionando ogni previsione sia sull'originale sia su quanto ha già tradotto, e durante l'addestramento vengono aggiornati i pesi dell'intero modello (encoder + decoder).

Transformer: una famiglia di architetture text-to-text

Oltre alla traduzione linguistica: molte attività possono essere formulate come testo in ingresso → testo in uscita



Quando parliamo di Transformer, parliamo di una famiglia di architetture, non di un unico modello.

La stessa idea di base può essere applicata a molti compiti diversi di tipo **text-to-text**.



Testo
in input



Rappresentazioni
contestuali



Testo
in output

Il modello legge un testo, ne costruisce rappresentazioni contestuali e genera un nuovo testo.

1



Riassunto

Condensa un testo lungo in una versione più breve.

Input:

articolo lungo



Output:

riassunto breve

2



Domanda e risposta

Riceve un testo e una domanda, e genera la risposta in linguaggio naturale.

Input:

brano + domanda



Output:

risposta

3



Parafrasi / riformulazione

Riscrive un testo con parole diverse mantenendo il significato.

Input:

frase originale



Output:

frase riformulata

4



Semplificazione del testo

Rende un testo più semplice e più facile da leggere.

Input:

testo complesso



Output:

testo semplificato

5



Correzione grammaticale e ortografica

Corregge errori di lingua mantenendo il contenuto.

Input:

testo con errori



Output:

testo corretto

6



Chatbot / assistente testuale

Genera una risposta coerente a partire da istruzioni o dialogo precedente.

Input:

richiesta utente



Output:

risposta del sistema



Nota didattica

Molti compiti NLP possono essere riformulati come text-to-text: si cambia il tipo di input e di output, ma resta la stessa logica generale.



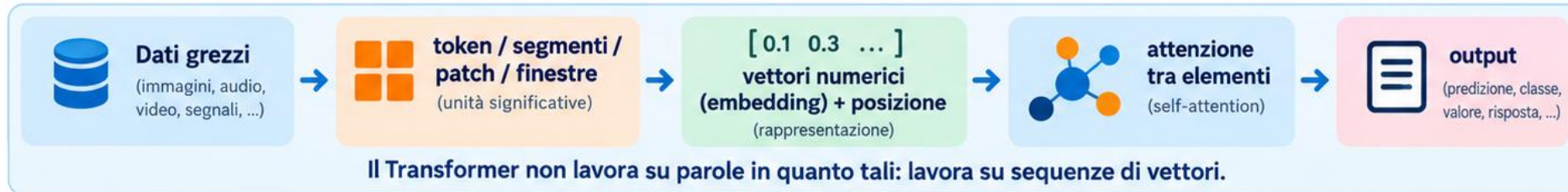
Idea chiave: la famiglia Transformer non serve solo a tradurre; può trasformare un testo in un altro testo utile per molti compiti diversi.

Transformer oltre il text-to-text

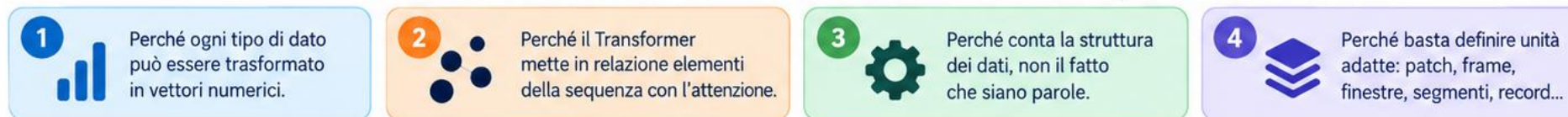
Esempi di applicazione a dati non testuali e perché funzionano



Parliamo di una famiglia di architetture Transformer: non servono solo per il testo. Funzionano anche con immagini, audio, video e altri dati sequenziali o segmentabili in parti.



Perché funziona anche se i dati non sono parole?



Idea chiave: il Transformer è generale perché elabora relazioni tra vettori in sequenza, non solo parole.

Tre famiglie di architetture Transformer

Encoder-only, Decoder-only, Encoder-Decoder: cosa fanno e quando si usano



Parliamo di una famiglia di architetture, non di un unico modello. La differenza principale è come trattano input e output.

1

Encoder-only

Legge il testo e ne costruisce rappresentazioni contestuali

Testo in input



Encoder



rappresentazione / etichetta / risposta breve



Scopo tipico:

- comprendere il testo
- classificare
- estrarre informazioni
- produrre embedding



Compiti tipici:

- classificazione del testo
- sentiment analysis
- named entity recognition
- ricerca semantica / embeddings



Idea chiave:

Ottimo quando serve capire l'input più che generare testo lungo.

Esempi: BERT, RoBERTa

2

Decoder-only

Genera il testo un token alla volta

Prompt / contesto



Decoder



testo generato



Scopo tipico:

- completare
- generare
- dialogare
- scrivere codice o testo



Compiti tipici:

- chatbot
- generazione di testo
- completamento di codice
- assistenti conversazionali



Idea chiave:

Ottimo quando l'obiettivo è produrre nuovo testo in modo autoregressivo.

Esempi: GPT, Llama

3

Encoder-Decoder

Legge un input e genera un nuovo output condizionato su di esso

Testo sorgente



Encoder



Decoder



nuovo testo



Scopo tipico:

- trasformare un testo in un altro testo
- usare l'input come guida esplicita



Compiti tipici:

- traduzione automatica
- riassunto
- parafrasi
- text-to-text



Idea chiave:

Ottimo quando c'è un input da comprendere e un output da generare a partire da esso.

Esempi: T5, BART

Confronto rapido



Encoder-only → capisce



Decoder-only → genera



Encoder-Decoder → trasforma



Messaggio da ricordare:

Le tre configurazioni usano la stessa idea di base dei Transformer, ma sono ottimizzate per scopi diversi: comprensione, generazione o trasformazione di testo.

Da dove viene il termine “Transformer”?

Origine del nome dell'architettura introdotta nel 2017



Parliamo di una famiglia di architetture, non di un singolo modello.

1 Nascita del nome

Il nome compare nel paper del 2017 'Attention Is All You Need'. Gli autori presentano una nuova architettura basata soprattutto su meccanismi di attention.



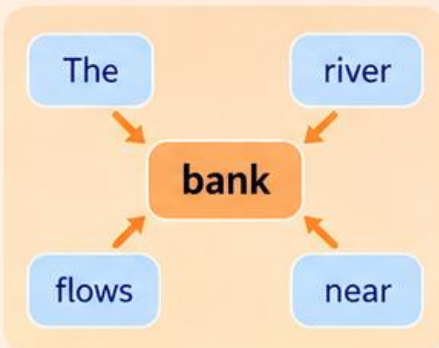
2 Perché si chiama Transformer?

Perché trasforma progressivamente le rappresentazioni dei dati: da token iniziali a rappresentazioni contestuali sempre più ricche.



3 Che cosa trasforma, in pratica?

Trasforma vettori numerici, non solo parole. Ogni layer aggiorna la rappresentazione di un elemento usando il contesto degli altri elementi della sequenza.



Lo stesso token (“bank”) può avere significati diversi a seconda del contesto.

4 Cosa NON significa

- Non si chiama così solo perché traduce una frase in un'altra lingua.
- La traduzione era il primo caso d'uso illustrato, ma l'idea è generale.
- La stessa famiglia di architetture è stata poi usata per molti compiti diversi.



Non è solo per la traduzione!



5 Curiosità storica

Secondo il racconto successivo di Jakob Ušzkoreit, il nome aveva anche un richiamo giocoso ai robot “Transformers”.



Qui il punto tecnico resta questo: il modello trasforma rappresentazioni tramite l'attention.



Nota: al primo passaggio c'è solo una rappresentazione vettoriale iniziale del token (embedding iniziale). Le rappresentazioni contestuali emergono dopo i passaggi di attention e vengono poi raffinate strato dopo strato.



Idea chiave:

“Transformer” indica un'architettura che trasforma rappresentazioni vettoriali in rappresentazioni contestuali sempre più ricche, mettendo in relazione gli elementi tramite attention.

L'evoluzione dei Transformer

Le principali direzioni di sviluppo di una famiglia di architetture, non di un singolo modello.



Parliamo di una **famiglia di architetture Transformer**: la stessa idea di base continua a evolvere lungo più direzioni contemporaneamente.

1

Multimodalità

Dal solo testo a combinazioni di testo, immagini, audio e video.



Testo



Immagini



Audio



Video

Esempi: VLM, captioning, visual question answering, modelli audio-video.

2

Contesti più lunghi e memoria

Finestre di contesto più ampie e meccanismi per ricordare più informazione utile.



Documenti lunghi



Cronologia



Memoria

Obiettivo: ragionare su documenti lunghi, conversazioni estese e codice complesso.

3

Efficienza e scalabilità

Più capacità con meno costo computazionale.



Chip



Velocità



Ingranaggi

Direzioni tipiche: sparse attention, quantizzazione, distillazione, Mixture of Experts (MoE).

4

Specializzazione e personalizzazione

Adattamento a domini, compiti o organizzazioni specifiche.



Target



Settore



Configurazione

Esempi: fine-tuning, instruction tuning, modelli verticali, uso di conoscenza aziendale.

5

Strumenti, agenti e azione

Il Transformer non solo genera testo: può usare strumenti esterni e orchestrare azioni.



Strumenti



Connessioni



Agenti

Esempi: tool use, function calling, recupero di documenti, agenti multi-step.

6

Affidabilità, allineamento e sicurezza

Maggiore controllo del comportamento e riduzione dei rischi.



Sicurezza



Valutazione



Guardrail

Direzioni: guardrail, valutazione, robustezza, mitigazione delle allucinazioni, sicurezza.



MESSAGGIO DA RICORDARE

I Transformer non si stanno evolvendo in una sola direzione:

stanno diventando più multimodali, più efficienti, più specializzati, più capaci di usare strumenti e più controllabili.

Dai dati
a un mondo
di possibilità.

Foundation model vs Frontier model

Che differenza c'è tra un modello fondazionale e un modello di frontiera?

Stessa
intelligenza,
prospettive
diverse.

Foundation model (modello fondazionale)



Un solo modello,
molte possibilità.

Categoria descrittiva: descrive come il modello è costruito e come si usa.

Origine: Stanford, 2021 – paper “On the Opportunities and Risks of Foundation Models”.

- addestrato su dati molto ampi
- tipicamente con auto-supervisione
- su larga scala
- progettato per essere adattato a molti compiti (prompting o fine-tuning)

Nota: Non riguarda solo il testo: anche audio, visione e immagini.

Esempi: Whisper, CLIP, generatori di immagini, GPT.

Tutti i 25 modelli della nostra timeline sono foundation model: da GPT-1 a GPT-6.1 Sol.

Frontier model (modello di frontiera)



Più capacità,
più responsabilità.

Categoria relativa e di policy: indica i modelli più capaci del momento.

Contesto: Il termine si afferma nel 2023 nel dibattito sulla governance dell'AI.

- modelli alla frontiera dello stato dell'arte
- definiti rispetto ai migliori disponibili oggi
- attenzione ai rischi avanzati
- non dice come il modello è costruito

Rischi tipici: cyber, biosicurezza, autonomia.

Esempi di contesto: Frontier AI Regulation, Frontier Model Forum, vertice di Bletchley Park.

3 differenze chiave

1 Asse



Foundation

natura del modello.

Frontier

livello relativo di capacità e rischio.

2 Stabilità



Foundation

etichetta stabile/permanente.

Frontier

etichetta mobile.

Esempio: GPT-4 era frontier nel 2023; GPT-6 Astra è frontier oggi.

3 Relazione tra insiemi

Foundation models

Frontier models

Quasi tutti i frontier sono foundation model, ma pochissimi foundation sono frontier.

gpt-oss-20b = foundation, non frontier.
modello ristretto per classificare spam
= né foundation né frontier.



Aggancio normativo: AI Act

1 La versione finale dell'AI Act non usa i termini “foundation model” o “frontier model”.

2 Parla di modelli di IA per finalità generali (GPAI): concetto vicino ai foundation model.

3 Parla anche di GPAI a rischio sistemico: concetto vicino ai frontier model.



Presunzione oltre 10²⁵ FLOP di calcolo in addestramento.

(FLOP = operazioni in virgola mobile, misura del calcolo usato per addestrare.)



Esempio vivo di governance di frontiera

OpenAI ha rallentato il rilascio di GPT-6 Astra perché le valutazioni interne superavano la soglia “Critical” di cybersicurezza del Preparedness Framework, con accesso alle capacità cyber avanzate riservato a organizzazioni verificate.



In sintesi

Foundation dice che cos'è un modello.

Frontier dice quanto è avanti — e quanto va sorvegliato — in questo momento.

Modelli
di oggi,
un domani
più sicuro.

NOTA I modelli tradizionali di machine learning e le reti neurali profonde, quando vengono addestrati secondo il paradigma convenzionale dell'apprendimento **supervisionato**, richiedono generalmente **dati etichettati**. Questo non vale per la fase di preaddestramento degli LLM. In questa fase, gli LLM utilizzano l'apprendimento **auto-supervisionato**, in cui il modello genera le proprie etichette a partire dai dati di input.

Apprendimento supervisionato, non supervisionato e auto-supervisionato

Qual è la differenza tra supervised, unsupervised e self-supervised learning?

Apprendimento supervisionato (Supervised learning)



- Usa dati etichettati da esseri umani
- Ogni esempio ha un input e una risposta corretta
- Il modello impara la relazione tra input e output
- Obiettivo: prevedere un'etichetta o un valore

Esempi:

- classificazione spam / non spam
- previsione del prezzo di una casa

Tipi tipici: classificazione, regressione

Etichette: sì, fornite dall'uomo

Apprendimento non supervisionato (Unsupervised learning)



- Usa dati non etichettati
- Non esiste una risposta corretta già nota
- Il modello cerca strutture, gruppi o pattern nascosti
- Obiettivo: scoprire come sono organizzati i dati

Esempi:

- segmentazione clienti
- clustering di documenti
- riduzione di dimensionalità

Tipi tipici: clustering, associazioni, riduzione di dimensionalità

Etichette: no

Apprendimento auto-supervisionato (Self-supervised learning)



Obiettivo
auto-generato
dai dati

- Parte da dati non etichettati
- Il modello crea i segnali di apprendimento dai dati stessi
- Le etichette non sono scritte da persone, ma ricavate automaticamente
- Obiettivo: imparare rappresentazioni generali utili

Esempi:

- prevedere il token/parola successiva
- ricostruire una parte mancante del testo o dell'immagine
- pretraining degli LLM

Uso tipico: preaddestramento di modelli foundation

Etichette: generate automaticamente dai dati

Confronto rapido

	Supervisionato	Non supervisionato	Auto-supervisionato
Dati di partenza	etichettati	non etichettati	non etichettati
Da dove arrivano le etichette?	dall'uomo	non ci sono	dal dato stesso
Obiettivo	predire	scoprire pattern	imparare rappresentazioni
Esempio classico	spam detection	customer clustering	next-token prediction



Punto chiave: l'auto-supervisione non è la stessa cosa dell'apprendimento non supervisionato. In entrambi i casi si parte spesso da dati non etichettati, ma nell'auto-supervisione il compito di apprendimento viene costruito automaticamente a partire dai dati.



In sintesi: supervisionato = etichette umane; non supervisionato = scoperta di pattern; auto-supervisionato = etichette generate dal dato stesso.

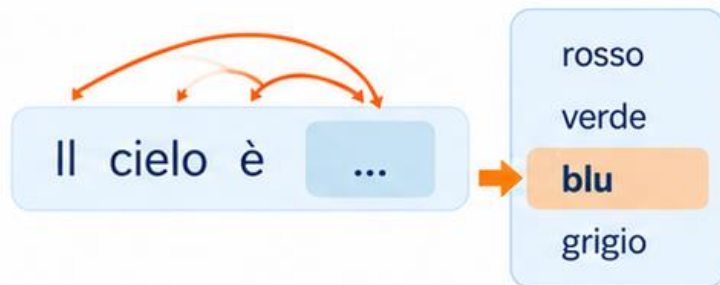
Esempi di apprendimento auto-supervisionato

Come il modello costruisce da solo il compito di apprendimento a partire dai dati



Nell'auto-supervisione si parte da **dati non etichettati**, ma il modello ricava automaticamente i segnali di apprendimento dai dati stessi.

1 Predizione del token/parola successiva



Il modello legge una sequenza di testo e impara a prevedere la parola o il token successivo.

Input: "Il cielo è ..." → **Target:** "blu"

Tipico negli LLM

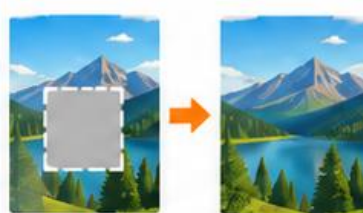
2 Ricostruzione di una parte mancante

Il modello impara a ricostruire **informazioni** mancanti nel testo o nelle immagini.

Testo (masked modeling)



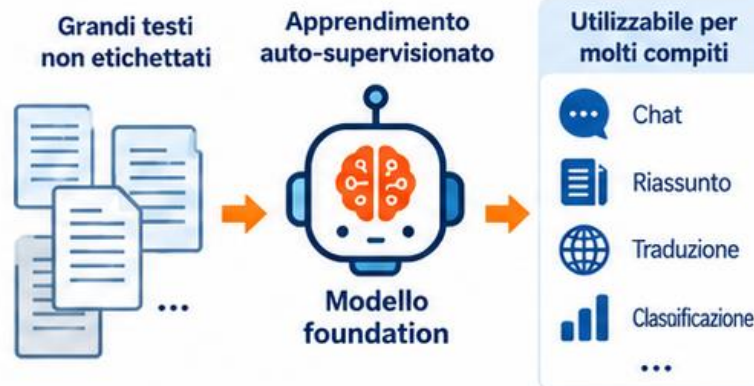
Immagine (inpainting)



Testo: "Roma è la capitale di [MASK]" → "Italia"
Immagine: patch nascosta → contenuto ricostruito

Ricostruzione / masked modeling

3 Pretraining degli LLM



Con compiti auto-generati su grandi quantità di testi, il modello apprende rappresentazioni linguistiche generali utili per molti compiti successivi.

Grandi testi non etichettati → apprendimento auto-supervisionato → modello foundation

Uso tipico

Cosa hanno in comune?



partono da **dati non etichettati**



le etichette non sono scritte da persone



il compito viene costruito automaticamente dai dati



Nel supervisionato, agli input sono associate risposte attese per il compito da imparare. Nell'auto-supervisionato, input e risposte attese vengono ricavati automaticamente dai dati stessi, per esempio nascondendone una parte e chiedendo al modello di prevederla.

Apprendimento supervisionato vs auto-supervisionato

Due modalità per addestrare un modello linguistico

1 Apprendimento supervisionato (es. traduzione con encoder-decoder)

Il modello viene addestrato su coppie di input e output di riferimento.

Esempio: traduzione

Frase sorgente (input)
This is an example



Traduzione di riferimento (target)
Das ist ein Beispiel

1 Input al modello

La frase sorgente viene processata dall'encoder; il decoder riceve la traduzione parziale corretta (teacher forcing).

2 Predizione del prossimo token

Il decoder genera la probabilità per il prossimo token (es. "Beispiel").

3 Calcolo dell'errore (loss)

Si confronta il token predetto con il token di riferimento.

4 Aggiornamento dei pesi

L'errore viene retropropagato e vengono aggiornati i pesi di TUTTO il modello (encoder, decoder, embedding e altri strati).



Risultato

Il modello impara a tradurre, cioè a generare l'output corretto dato l'input, minimizzando l'errore sulle coppie di addestramento.

2 Apprendimento auto-supervisionato (es. modellazione del prossimo token)

Il modello viene addestrato su grandi quantità di testo non etichettato, creando da solo l'obiettivo di apprendimento.

Esempio: modellazione del prossimo token

Testo di input (senza etichette)
The cat sits on the



Token da predire (obiettivo creato dal modello)
mat

1 Creazione dell'obiettivo

Si prende un testo e si maschera (o si rimuove) il token successivo: il modello deve predirlo a partire dal contesto precedente.

2 Predizione del prossimo token

Il modello calcola la distribuzione di probabilità per il token mancante (es. "mat").

3 Calcolo dell'errore (loss)

Si confronta il token predetto con il token reale (del testo originale).

4 Aggiornamento dei pesi

L'errore viene retropropagato e vengono aggiornati i pesi dell'intero modello (nelle architetture decoder-only: tutti i layer; nei modelli encoder-decoder usati in auto-supervisionato: entrambi).



Risultato

Il modello impara la struttura, la grammatica e il significato della lingua da grandi quantità di testo, senza etichette, acquisendo conoscenze generali utili per molti compiti.



Nota importante

In entrambi i casi, ciò che viene addestrato è il **modello completo** (encoder e decoder, quando presenti), non solo il decoder.

L'errore è calcolato sull'output del decoder (o sul token mancante), ma tramite backpropagation vengono aggiornati tutti i pesi del modello.

La prima fase di addestramento di un LLM è nota anche come *preaddestramento* (*pretraining*) e produce un LLM iniziale preaddestrato, spesso chiamato *modello di base* o *modello fondazionale*. Un esempio tipico è GPT-3, il precursore del modello originariamente offerto in ChatGPT. Questo modello è in grado di completare un testo, ossia di terminare una frase scritta solo in parte e fornita da un utente. Possiede inoltre capacità limitate di apprendimento *few-shot*: ciò significa che può imparare a svolgere nuovi compiti basandosi soltanto su pochi esempi, anziché richiedere grandi quantità di dati di addestramento.

Dopo aver ottenuto un LLM preaddestrato mediante l'addestramento su grandi insiemi di dati testuali, durante il quale il modello impara a prevedere la parola successiva nel testo, possiamo addestrarlo ulteriormente su dati etichettati. Questo ulteriore addestramento è noto come *fine-tuning*.

Le due categorie più diffuse di fine-tuning degli LLM sono il *fine-tuning sulle istruzioni* (*instruction fine-tuning*) e il *fine-tuning per la classificazione* (*classification fine-tuning*). Nel fine-tuning sulle istruzioni, il dataset etichettato è costituito da coppie di istruzioni e risposte: per esempio, una richiesta di traduzione di un testo accompagnata dalla sua traduzione corretta. Nel fine-tuning per la classificazione, il dataset etichettato è costituito da testi e dalle corrispondenti etichette di classe: per esempio, messaggi di posta elettronica associati alle etichette «spam» e «non spam».

Vedremo le implementazioni in codice per il preaddestramento e il fine-tuning di un LLM e, dopo aver preaddestrato un LLM di base, approfondiremo le caratteristiche specifiche sia del fine-tuning sulle istruzioni sia di quello per la classificazione.

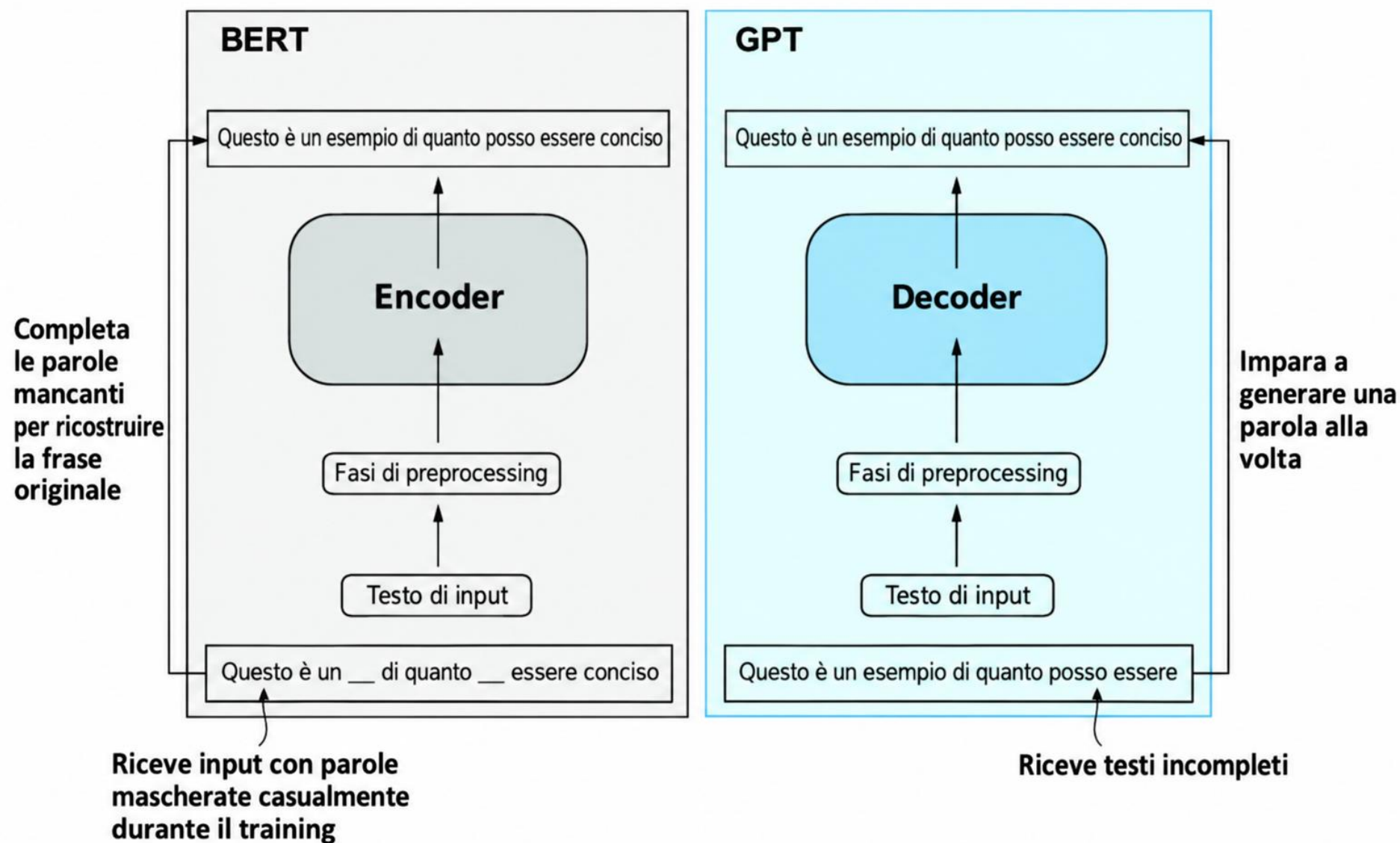
Le successive varianti dell'architettura Transformer, come **BERT** (acronimo di *Bidirectional Encoder Representations from Transformers*) e i diversi modelli **GPT** (acronimo di *Generative Pretrained Transformers*), hanno sviluppato questo concetto per **adattare l'architettura a compiti differenti**.

BERT, basato sul sottomodulo *encoder* del Transformer originale, si distingue da GPT per il metodo di addestramento. Mentre GPT è progettato per compiti generativi, BERT e le sue varianti si specializzano nella **predizione di parole mascherate**: il modello predice le parole mascherate o nascoste in una determinata frase, come illustrato nella figura seguente.

Questa particolare strategia di addestramento rende BERT particolarmente efficace nei compiti di classificazione del testo, tra cui l'analisi del sentiment e la categorizzazione dei documenti. Come esempio applicativo, X (precedentemente Twitter) utilizza BERT per rilevare contenuti tossici.

GPT, invece, si basa sulla componente *decoder* dell'architettura Transformer originale ed è progettato per compiti che richiedono la **generazione di testo**. Tra questi rientrano la traduzione automatica, il riassunto di testi, la scrittura narrativa, la generazione di codice informatico e altro ancora. I modelli GPT, progettati e addestrati principalmente per completare testi, mostrano anche una **notevole versatilità**. Sono in grado di svolgere compiti sia in modalità **zero-shot** sia in modalità **few-shot**.

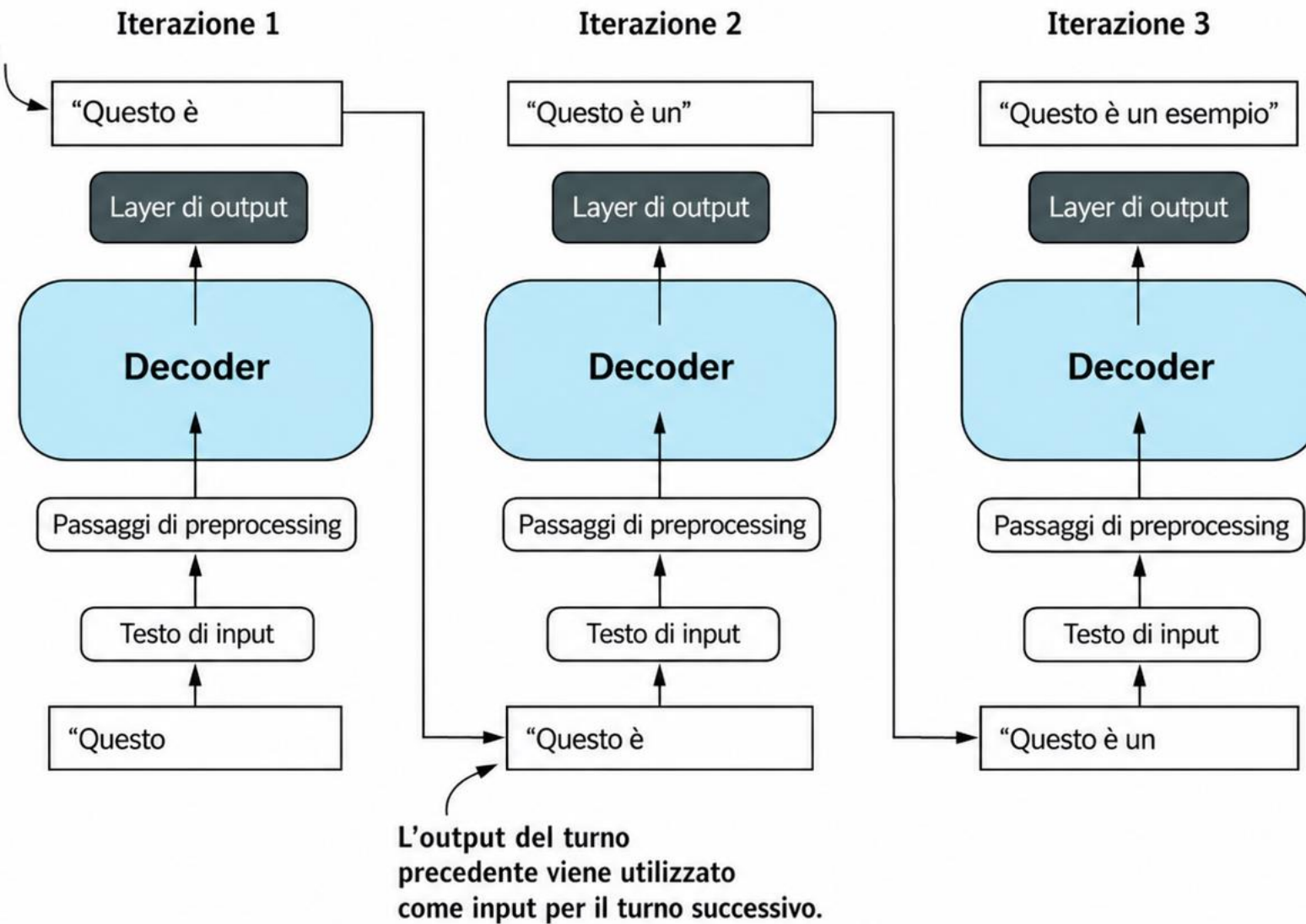
Lo **zero-shot learning** indica la capacità di generalizzare a compiti completamente nuovi, senza disporre di esempi specifici precedenti. Il **few-shot learning**, invece, consiste nell'apprendere da un numero molto limitato di esempi che l'utente fornisce nell'input, come illustrato nella figura seguente.



Rappresentazione visiva dei sottomoduli encoder e decoder del Transformer.

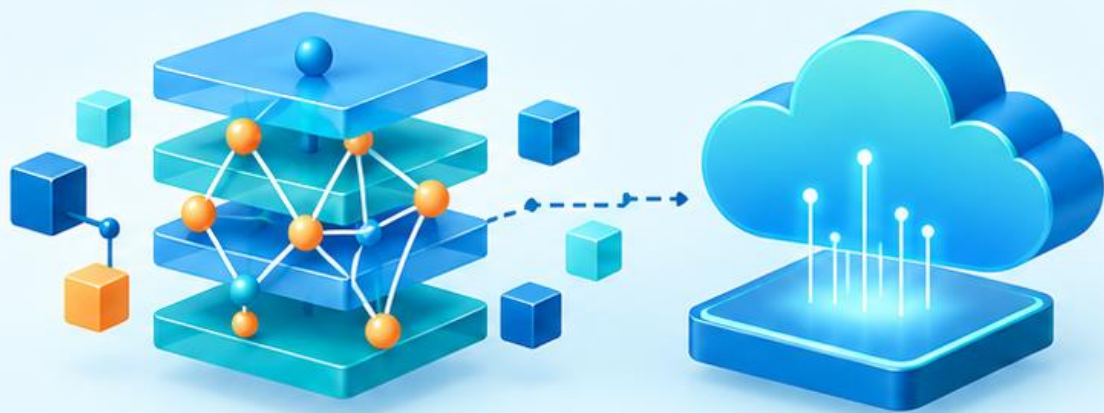
A sinistra, il segmento **encoder** esemplifica gli LLM di tipo **BERT**, che si concentrano sulla **predizione di parole mascherate** e sono utilizzati principalmente per attività come la **classificazione del testo**. A destra, il segmento **decoder** mostra gli LLM di tipo **GPT**, progettati per attività **generative** e per produrre **sequenze di testo coerenti**.

Genera la parola successiva in base al testo di input

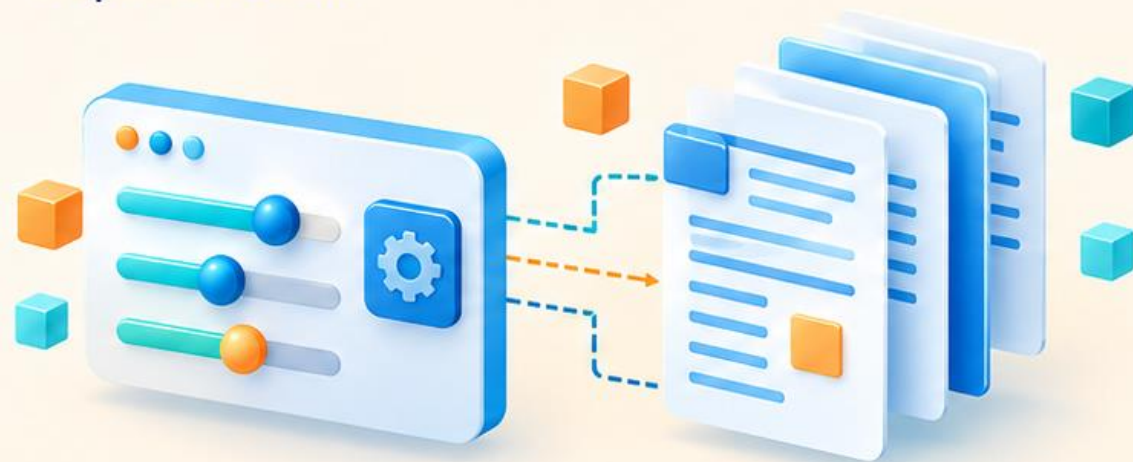


L'architettura di GPT utilizza solo la parte decoder del Transformer originale. È progettata per l'elaborazione unidirezionale, da sinistra a destra, e risulta particolarmente adatta alla generazione di testo e ai compiti di predizione della parola successiva, producendo testo in modo iterativo, una parola alla volta.

La natura pre-addestrata di questi modelli li rende incredibilmente versatili per un successivo **fine-tuning** su compiti a valle (*downstream task*), motivo per cui sono noti anche come modelli base o *foundation model*. Il pre-addestramento degli LLM richiede l'accesso a risorse considerevoli ed è molto costoso: per esempio, il costo di pre-addestramento di GPT-3 è stimato in **4,6 milioni di dollari** in crediti di cloud computing (<https://mng.bz/VxEW>).

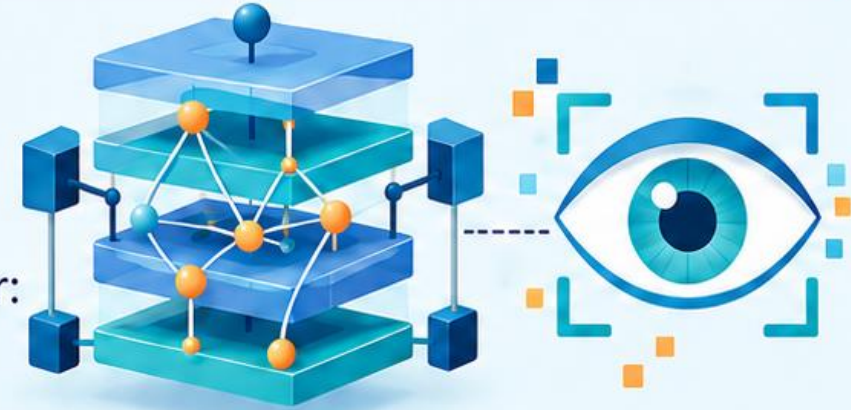


La buona notizia è che molti LLM pre-addestrati, disponibili come modelli **open source**, possono essere utilizzati come strumenti di uso generale per scrivere, estrarre e modificare testi che non facevano parte dei dati di addestramento. Inoltre, gli LLM possono essere sottoposti a **fine-tuning** su compiti specifici con dataset relativamente più piccoli, riducendo le risorse computazionali necessarie e migliorando le prestazioni.



TRANSFORMER E LLM A CONFRONTO

Gli LLM attuali si basano sull'architettura Transformer. Per questo motivo, nella letteratura i termini «Transformer» e «LLM» vengono spesso utilizzati come sinonimi. Tuttavia, non tutti i Transformer sono LLM, poiché possono essere impiegati anche nella **computer vision**. Inoltre, non tutti gli LLM sono Transformer: esistono infatti LLM basati su architetture ricorrenti e convoluzionali.



La motivazione principale di questi approcci alternativi è migliorare l'efficienza computazionale degli LLM. Resta da vedere se tali architetture alternative potranno competere con le capacità degli LLM basati sui Transformer e se verranno adottate nella pratica.



Nel corso useremo il termine "LLM" per riferirci agli LLM basati sui trasformatori e simili a GPT.



GPT-3 è un modello linguistico autoregressivo: calcola la probabilità del prossimo token sulla base dei token che lo precedono. Il paper originale lo definisce esplicitamente così.

Che cosa significa “autoregressivo”

Durante la **generazione**, il procedimento si ripete:

1. Il modello riceve una sequenza di token e calcola la **distribuzione di probabilità del prossimo token**.
2. Viene selezionato un token secondo la procedura di decodifica: per esempio, tramite campionamento oppure scegliendo quello più probabile.
3. Il token selezionato viene **aggiunto alla sequenza**, che diventa il contesto per il passo successivo.

Quindi, **anche ciò che il modello ha già generato contribuisce a determinare ciò che genererà dopo**.

Formalmente, a ogni passo stima:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t)$$

cioè la probabilità del token successivo, condizionata ai token già presenti.

Un esempio semplificato

Per leggibilità, qui uso parole intere; i token effettivi possono essere anche parti di parola.

Passo	Testo disponibile	Possibile elemento generato	
1	Il gatto	dorme	
2	Il gatto dorme	sul	
3	Il gatto dorme sul	divano	

Il collegamento con la self-attention

GPT-3 usa **self-attention causale**: la rappresentazione di un token può dipendere da quel token e da quelli precedenti, ma **non da quelli successivi**. La rappresentazione all'ultima posizione disponibile serve a prevedere il prossimo token.

Questo distingue il suo funzionamento da quello di **BERT**, che costruisce rappresentazioni bidirezionali usando il contesto sia a sinistra sia a destra della posizione considerata.

Anche molti altri LLM sono autoregressivi. GPT-3 condivide questo funzionamento con numerosi modelli generativi. L'autoregressione è un approccio molto diffuso, ma "LLM" e "modello autoregressivo" non sono sinonimi: esistono anche modelli linguistici che funzionano diversamente. [arxiv.org](#)

Le principali famiglie

Famiglia	Esempi	Autoregressivi?	Come funzionano
Modelli generativi con decoder causale	GPT-3, Llama 3.1	Sì	Generano il prossimo token sulla base del contesto disponibile, includendo i token già generati.
Modelli con mascheramento e encoder bidirezionale	BERT	No, nel funzionamento standard	Imparano a ricostruire token nascosti usando il contesto sia a sinistra sia a destra.
Modelli encoder-decoder	T5	Sì, nella generazione del decoder	L'encoder legge l'input in modo bidirezionale; il decoder produce l'output autoregressivamente.
Modelli linguistici a diffusione	LLaDA	Usano un'alternativa alla generazione autoregressiva classica	Possono costruire il testo attraverso più passaggi di ricostruzione di una sequenza mascherata.

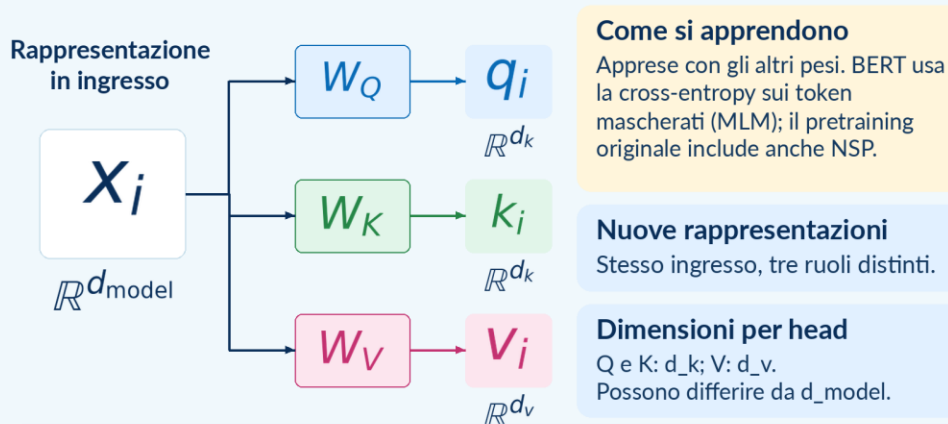
Query, Key e Value nella Self-Attention dei Transformer 1/2

GPT / BERT

Tre rappresentazioni dello stesso token per integrare le informazioni del contesto

1 Dall'ingresso ai tre vettori

Da ogni rappresentazione x_i si ottengono tre proiezioni apprese.



Il vantaggio è non obbligare lo stesso vettore a svolgere tre compiti diversi nello stesso modo. Con Q, K e V, il modello può imparare separatamente come cercare informazioni, come rendere un token individuabile e quali informazioni trasferire.

Rielaborazione da Sebastian Raschka, "Self-attention in LLMs".

2 I ruoli di Query, Key e Value

Tre funzioni distinte; le domande sono metafore didattiche.



Query (Q)

«Che cosa cerco?»

Confronto con le Key.

Esempio: "aveva"

«Chi è il soggetto sottinteso?»



Key (K)

«Come posso essere individuato?»

Caratteristiche per il confronto.

Esempio: "gatto"

Possibile soggetto compatibile.



Value (V)

«Quale informazione trasferisco?»

Contenuto da combinare con i pesi.

Esempio: "gatto"

Informazioni sulla rappresentazione di "gatto".

3 Due esempi illustrativi

Le frecce indicano: token $\rightarrow$ vettore. Query e Key servono al confronto; i Value forniscono il contenuto.

Esempio 1 • Verbo e soggetto

TOKEN CORRENTE: "aveva"

Il **gatto** ha bevuto il latte perché **aveva** sete.

Key di "gatto"

Possibile soggetto compatibile.

Value di "gatto"

Informazioni sul soggetto.

Query di "aveva"

Chi è il soggetto sottinteso?

Stiamo costruendo il context vector di "aveva".

Esempio 2 • Pronome e antecedente

TOKEN CORRENTE: "che"

Maria ha comprato un **libro** **che** parla di astronomia.

Key di "libro"

Possibile antecedente compatibile.

Value di "libro"

Informazioni sull'antecedente.

Query di "che"

Qual è il mio antecedente?

Stiamo costruendo il context vector di "che".

SELF-ATTENTION BIDIREZIONALE, COME IN BERT

Non è la self-attention causale dei modelli GPT-like.

Ogni token può usare tutti i token della sequenza, a sinistra e a destra, incluso sé stesso.

Tokenizzazione semplificata: una parola per token. Esempi didattici, non attenzioni misurate.

Fonti: Vaswani et al. (2017), §3.2; Devlin et al. (2019), §3; S. Raschka, "Self-attention in LLMs" (rielaborazione).

4 Come si combina tutto

Dal token corrente al context vector: schema di una singola head, senza maschera causale.

1 Query del token corrente

La Query proviene dal token i . Key e Value provengono da tutti i token j , incluso i .

q_i

Tutte le posizioni sono ammesse:

$j = 1, \dots, T$

2 Compatibilità Query-Key

Un confronto appreso, non una semplice somiglianza tra parole.

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Il divisore scala lo score; non porta i vettori a norma 1.

3 Softmax sugli score

Ogni score è confrontato con tutti gli score della stessa Query.

$$\alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{\ell=1}^T e^{s_{i\ell}}}$$

$\alpha_{ij} \geq 0, \quad \sum_j \alpha_{ij} = 1$

4 Somma pesata dei Value

Ogni peso moltiplica il suo Value. Si sommano i contributi di tutti i Value, non le Key.

$$z_i = \sum_{j=1}^T \alpha_{ij} V_j$$

Non si seleziona un unico token.

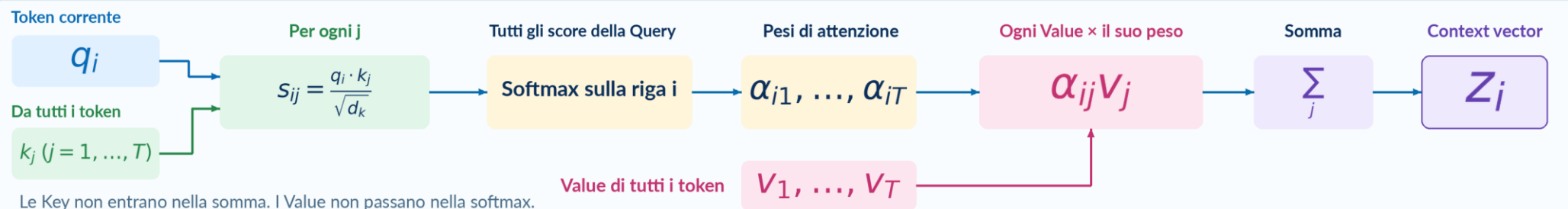
5 Context vector della head

Il risultato è il context vector del token corrente per questa head.

$z_i \in \mathbb{R}^{d_v}$

Non è ancora l'output dell'intero blocco Transformer.

Il flusso corretto: confronto e contenuto restano separati



Esempio: il token corrente è "che"

Maria ha comprato un libro **che** parla di astronomia .

Anche i token a destra, come "parla" e "astronomia", possono contribuire al risultato.

Da una head al blocco

I risultati delle head vengono concatenati e proiettati. Il blocco comprende anche connessioni residue, normalizzazione e rete feed-forward.

SELF-ATTENTION BIDIREZIONALE, COME IN BERT

Non è la self-attention causale dei modelli GPT-like.

Ogni token può usare tutti i token della sequenza, a sinistra e a destra, incluso sé stesso.

Sommario del capitolo 1

Evoluzione e addestramento

Gli LLM hanno trasformato il campo dell'elaborazione del linguaggio naturale, che in precedenza si basava soprattutto su sistemi con regole esplicite e metodi statistici più semplici. Il loro avvento ha introdotto nuovi approcci basati sul deep learning, consentendo progressi nella comprensione, generazione e traduzione del linguaggio umano.

Gli LLM moderni vengono addestrati in due fasi principali:

1. Prima, sono preaddestrati su un ampio corpus di testi non etichettati, utilizzando come etichetta la parola successiva da predire in una frase (**come vedremo presto, il task di previsione della parola successiva degli LLM, durante il loro pre-addestramento, consiste nel prevedere la prossima parola che segue il blocco di input**)
2. Quindi, sono fine-tuned su un dataset mirato, più piccolo ed etichettato, per seguire istruzioni o svolgere compiti di classificazione.

Una volta completato il preaddestramento, il modello di base (foundation model) può essere adattato mediante fine-tuning in modo più efficiente a diversi compiti applicativi successivi (downstream).

Gli LLM sottoposti a fine-tuning su dataset personalizzati possono superare gli LLM generalisti in compiti specifici.

Architettura e capacità

Gli LLM si basano sull'architettura Transformer. La sua idea centrale è un meccanismo di attenzione che consente al modello di accedere selettivamente all'intera sequenza di input mentre genera l'output una parola alla volta.

L'architettura Transformer originale comprende un encoder per analizzare il testo e un decoder per generarlo.

Gli LLM per la generazione di testo e l'esecuzione di istruzioni, come GPT-3 e ChatGPT, utilizzano soltanto moduli decoder, semplificando l'architettura.

Per il preaddestramento degli LLM sono essenziali grandi dataset composti da miliardi di parole.

Sebbene il compito generale di preaddestramento dei modelli di tipo GPT consista nel predire la parola successiva in una frase, questi LLM mostrano proprietà emergenti, come la capacità di classificare, tradurre o riassumere testi.